

END OF STUDIES PROJECT REPORT

By

Youssef HASNAOUI

Design and Implementation of a Debugger with Instruction Trace Capabilities

Professional Supervisor:

Mohamed HAMROUNI

Academic Supervisor:

Faten Salem

Project Elaborated Within STMicroelectronics



Dedications

I dedicate this work to my family and friends.

*To my mother **Monia Sellami**, your love and support means so much to me in every step of my journey. You're the best.*

*To my father **Makram Hasnaoui**, thank you so much for all the sacrifices you've made for my sake. I could never be here without you.*

*To my sister **Aicha**, your kindness and your belief in me have been a constant source of inspiration, and for that, I am very grateful.*

To my friends, you have my most sincere gratitude for your support, your guidance and your company throughout this journey.

Thank you,

Youssef HASNAOUI

Acknowledgements

I would like to sincerely thank everyone who had a hand in the development of this project.

*Firstly, I want to express my utmost appreciation to **Mohamed HAMROUNI**, his guidance and support have been invaluable, and have kept me motivated and determined to give my best effort.*

*I would also like to thank my professor **Faten SALEM** for providing the essential foundation to pursue this domain of expertise and for her captivating curiosity, which inspired me to continuously expand the boundaries of my knowledge.*

Last but not least, to every single person that contributed to the successful completion of this project with their constant support and motivation, you have my utmost thanks.

Contents

List of Figures	viii
List of Tables	ix
List of Acronyms	x
Introduction	1
1 Context & Problem Statement	2
1.1 Host Organization	3
1.1.1 STMicroelectronics	3
1.1.2 Activity Sectors	3
1.1.3 Global Presence	3
1.1.4 ST Tunis	4
1.1.5 Application & Product Support Team	4
1.2 Project Context	5
1.2.1 Problem Statement	5
1.2.2 State of the Art	6
1.2.3 Critique of Current State of the Art	6
1.2.4 Proposed Solution	7
1.3 Conclusion	7
2 Theory and Key Concepts	8
2.1 Debugging Fundamentals	9
2.1.1 Bugs, Specifications and Defects	9
2.1.2 Anatomy of a Debugger	9
2.1.3 The Debug Adapter Protocol	10
2.1.4 The GDB Remote Serial Protocol	10
2.2 Debug Primitives	11
2.2.1 Run Control	11
2.2.2 Breakpoints: Software and Hardware	11
2.2.3 State Inspection: Registers, Memory and Variables	12
2.2.4 Call Stacks and Unwinding	12

2.2.5	Disassembly	12
2.3	Classes of Debugging	12
2.3.1	Intrusive and Non-Intrusive Debugging	12
2.3.2	Live and Post-Mortem Debugging	13
2.3.3	Run-Control and Trace-Based Debugging	13
2.4	Program Representation	13
2.4.1	The Embedded Build Model	13
2.4.2	The ELF Format	14
2.4.3	DWARF Debug Information	14
2.5	Microcontrollers	15
2.5.1	Device Description: CMSIS-SVD	15
2.6	The Arm Cortex-M Architecture	16
2.6.1	Arm Architecture	16
2.6.2	Register Model and the Thumb Instruction Set	16
2.6.3	Exception Model and Vector Table	16
2.6.4	System Address Map	16
2.6.5	AMBA Buses and their Roles in Debug	17
2.7	The Arm Debug Architecture	18
2.7.1	The Arm Debug Interface	18
2.7.2	Debug Port: JTAG and SWD	19
2.7.3	Access Ports and the Memory Access Port	19
2.7.4	Component Identification and ROM Tables	20
2.7.5	Halting Debug Control	20
2.7.6	Reset Types and Vector Catch	21
2.7.7	Breakpoint and Watchpoint Units	21
2.8	The CoreSight Trace Architecture	22
2.8.1	Trace Sources, Links and Sinks	22
2.8.2	Instruction, Data and Instrumentation Trace	22
2.8.3	Trace Source: the Embedded Trace Macrocell	23
2.8.4	The CoreSight Trace Formatter	25
2.8.5	Off-Chip Trace Sinks: TPIU and SWO	26
2.8.6	On-Chip Trace Sinks: the Trace Memory Controller	27
2.9	Debug and Trace Infrastructure in STM32 Microcontrollers	28
2.9.1	Debug Access and Trace Capabilities	28

2.9.2	The DBGMCU Component	29
2.10	Conclusion	30
3	Objectives Specification and Work Environment	31
3.1	Project Objectives and Deliverables	32
3.1.1	Scope	32
3.2	Requirements Analysis	33
3.2.1	Functional Requirements	33
3.2.2	Non-Functional Requirements	34
3.2.3	Traceability to Deliverables	34
3.3	Hardware Environment	34
3.3.1	Microcontroller: STM32H7S3L8	34
3.3.2	Development Board: NUCLEO-H7S3L8	35
3.3.3	Debug Probe: STLINK-V3EC	35
3.3.4	Trace Resources Available on the Target	36
3.4	Software Environment	36
3.4.1	Development Environment	36
3.4.2	External Software Components	37
3.5	Conclusion	37
4	Architecture	38
4.1	System Overview	39
4.1.1	Global Architecture	39
4.1.2	Design Rules	39
4.2	Instruction Trace Architecture	40
4.2.1	Pipeline Stages	40
4.2.2	Generation and Capture	40
4.2.3	Extraction	41
4.2.4	Decoding	41
4.2.5	Reconstruction and Source Correlation	41
4.2.6	Trace Data Model	41
4.2.7	Trace Lifecycle	42
4.3	Debugger Architecture	42
4.3.1	Layers	42
4.3.2	The Debug Context	43

4.3.3	Providers	43
4.3.4	Division of Work Between the Debugger Library and the Debug Server	43
4.3.5	Concurrency	45
4.3.6	Protocol Layer	46
4.3.7	Frontend	46
4.4	System-Wide Rules	46
4.4.1	Error Model	46
4.4.2	Module Boundaries	46
4.5	Conclusion	47
5	Implementation	48
5.1	Development Chronology	49
5.2	The Application Note	49
5.3	Instruction Trace Implementation	50
5.3.1	CoreSight Trace Drivers	50
5.3.2	Trace Extraction	51
5.3.3	Trace Decoding	53
5.3.4	Static Program Analysis	53
5.3.5	Reconstruction Transforms	54
5.3.6	Delivery to the Frontend	55
5.4	The Debugger	55
5.4.1	OpenOCD as an In-Process Library	55
5.4.2	LLDB Integration	57
5.4.3	Session Lifecycle	58
5.4.4	Reset and State Resynchronization	58
5.4.5	Breakpoints	59
5.4.6	Memory Access and Live Observation	60
5.4.7	Peripheral Inspection	61
5.4.8	Disassembly, Variables and Expressions	61
5.4.9	Protocol Server and Custom Extensions	62
5.4.10	The Frontend Extension	62
5.4.11	Portability and Provenance	63
5.5	Testing and Validation	64
5.5.1	Test Strategy	64

5.5.2	Hardware Test Bench	65
5.5.3	Trace Correctness Validation	66
5.5.4	Defects Found on Hardware	67
5.5.5	Results	68
5.5.6	Limitations and Future Work	69
5.6	Conclusion	70
Conclusion		71
Bibliography		72

List of Figures

1.1	STMicroelectronics Activity Sectors	3
1.2	STMicroelectronics Worldwide Presence	4
1.3	STMicroelectronics Tunis	5
2.1	Conceptual decomposition of a debugger	10
2.2	Software and hardware breakpoint mechanisms	11
2.3	The M-profile system address map	17
2.4	Structure of a Debug Access Port	19
2.5	Trace elements generated for a short loop	25
2.6	Structure of a formatter frame	26
3.1	Debug and trace infrastructure of the target	35
4.1	Global architecture of the solution	39
4.2	The four-stage instruction trace pipeline	40
4.3	The trace lifecycle	42
4.4	Module dependency graph	43
4.5	Division of work between the debugger library and the debug server	45
5.1	Extraction sequence	52
5.2	From bytes to elements	54
5.3	Command queue and wake mechanism	57
5.4	Reset and resynchronization	59
5.5	Breakpoint placement decision	60
5.6	Peripheral write and observation	62
5.7	The trace views	63
5.8	Memory and peripheral views	64

List of Tables

1.1	Embedded Systems IDE Vendors	6
1.2	Debugger Features	6
1.3	Debug Probe Limitations	7
2.1	Arm Mapping Symbols	14
2.2	DWARF constructs and the operations they support	15
2.3	AMBA buses and their roles in debug and trace	18
2.4	Registers governing halting debug	21
2.5	Types of trace	22
2.6	Kinds of trace element	24
2.7	Trace Memory Controller configurations	27
2.8	Trace Memory Controller operating modes	27
2.9	On-chip and off-chip trace capture	28
2.10	Trace capability of the STM32 families surveyed	29
3.1	Deliverables	32
3.2	Scope of the three deliverables	32
3.3	Functional requirements	33
3.4	Non-functional requirements	34
3.5	Traceability of requirements to deliverables	34
3.6	Trace components of the target device	36
3.7	Development environment	36
3.8	External software components	37
5.1	Registered protocol requests by area	63
5.2	Test inventory	65
5.3	Defects found on hardware	67
5.4	Verification of the requirements	68
5.5	Measured figures for one capture	69
5.6	Limitations of the trace pipeline	70

List of Acronyms

- **ADI** = Arm **D**ebug **I**nterface
- **AHB** = Advanced **H**igh-performance **B**us
- **AMBA** = Advanced **M**icrocontroller **B**us Architecture
- **APB** = Advanced **P**eripheral **B**us
- **API** = Application **P**rogramming **I**nterface
- **ARM** = Advanced **R**ISC **M**achine
- **ATB** = Advanced **T**race **B**us
- **AXI** = Advanced e**X**tensible **I**nterface
- **CFA** = Canonical **F**rame Address
- **CMSIS** = Cortex **M**icrocontroller **S**oftware **I**nterface **S**tandard
- **CMSIS-DAP** = The standard interface between a debug probe and a host, defined by Arm as part of CMSIS
- **CTI** = Cross-**T**rigger **I**nterface
- **DAP** = **D**ebug **A**ccess **P**ort (Arm Debug Interface)

- **DBGMCU** = **Debug Microcontroller Unit**
- **DEMCR** = **Debug Exception and Monitor Control Register**
- **DFSR** = **Debug Fault Status Register**
- **DHCSR** = **Debug Halting Control and Status Register**
- **DIE** = **Debugging Information Entry**
- **DWARF** = **Debugging With Attributed Record Formats**
- **DWT** = **Data Watchpoint and Trace**
- **ELF** = **Executable and Linkable Format**
- **ETB** = **Embedded Trace Buffer**
- **ETF** = **Embedded Trace FIFO**
- **ETM** = **Embedded Trace Macrocell**
- **ETR** = **Embedded Trace Router**
- **FPB** = **Flash Patch and Breakpoint**
- **GCC** = **GNU Compiler Collection**

- **GDB** = **GNU Debugger**
- **GNU** = **GNU's Not Unix**
- **IAR** = **Ingenjöröfirma Anders Rundgren**
- **IDE** = **Integrated Development Environment**
- **IEEE** = **Institute of Electrical and Electronics Engineers**
- **ITM** = **Instrumentation Trace Macrocell**
- **JSON** = **JavaScript Object Notation**
- **JTAG** = **Joint Test Action Group**
- **LLVM** = **Low Level Virtual Machine**
- **MCU** = **Microcontroller Unit**
- **MEM-AP** = **Memory Access Port**
- **MIPI** = **Mobile Industry Processor Interface**
- **PPB** = **Private Peripheral Bus**
- **RAM** = **Random Access Memory**

- **ROM** = **Read-Only Memory**
- **RSP** = **Remote Serial Protocol**
- **SB** = **Scripting Bridge**
- **SVD** = **System View Description**
- **SWD** = **Serial Wire Debug**
- **SWO** = **Serial Wire Output**
- **TFBGA** = **Thin Fine-pitch Ball Grid Array**
- **TMC** = **Trace Memory Controller**
- **TPIU** = **Trace Port Interface Unit**
- **UART** = **Universal Asynchronous Receiver Transmitter**
- **VCP** = **Virtual COM Port**
- **XML** = **eXtensible Markup Language**

Introduction

Embedded systems operate under real-time constraints. They interact directly with hardware peripherals and with asynchronous external events. Both properties make them difficult to observe and to debug. Traditional techniques such as serial logging or software breakpoints are often insufficiently informative, and they are intrusive enough to perturb the behavior under examination. Modern Arm-based microcontrollers address these limitations by integrating hardware debug and trace capabilities through the Arm CoreSight architecture, which permits non-intrusive visibility into instruction execution, memory accesses and system activity.

STM32 microcontrollers expose a substantial subset of these capabilities. Despite the sophistication of the underlying hardware, the features remain locked behind proprietary tooling, or surface only as the limited workflows that commercial development environments choose to expose. Within mainstream open-source debugging environments, instruction trace is rarely treated as a first-class debugging primitive, and exists instead as a fragmented collection of low-level mechanisms requiring manual integration.

Using instruction trace in practice involves extracting raw trace data through custom scripts, decoding packets through a standalone framework such as OpenCSD, and relying on visualization or analysis tools disconnected from the debugging workflow itself. The resulting process lacks standardization, interoperability and accessibility. Advanced trace-assisted debugging is therefore placed outside the reach of many embedded developers, despite the hardware support already present on the devices they use.

This project addresses that gap through a study of the trace and debug infrastructure available on STM32 microcontrollers, spanning CoreSight components, transport protocols, debug probes, software tooling and host-side integration mechanisms. The work then proposes a practical and reproducible workflow for instruction trace capture, decoding and interpretation, designed to integrate with existing environments while remaining independent of proprietary abstractions.

The report is organized in five chapters. The first will present the host organization and will state the problem the project addresses, together with a survey of what existing tools provide. The second will establish the theory the later chapters rely upon, covering debugging fundamentals, program representation, the Arm debug architecture and the CoreSight trace architecture. The third will state the objectives, the requirements derived from them and the working environment. The fourth will describe the architecture of the solution, its decomposition into components and the reasons behind each decision. The fifth will describe the realization of the three deliverables, the tests performed and the results obtained.

CONTEXT & PROBLEM STATEMENT

Contents

1	Host Organization	3
2	Project Context	5
3	Conclusion	7

Introduction

This chapter states the setting of the project and the problem it addresses. It presents the host organization, then examines what existing development tools offer for instruction trace, what they withhold, and what the hardware makes possible regardless.

1.1 Host Organization

1.1.1 STMicroelectronics

STMicroelectronics (ST) is a global leader in the semiconductor industry, providing innovative solutions across various markets, including automotive, industrial, personal electronics, and communications equipment.

Founded in 1987 through the merger of SGS Microelettronica of Italy and Thomson Semiconducteurs of France, ST has grown to become one of the largest semiconductor companies in the world.

1.1.2 Activity Sectors

ST's mission is to be the undisputed leader in the semiconductor market, delivering sustainable and innovative solutions that make a positive impact on people's lives. The company's vision is to create technology that enables a more intelligent and connected world, driving progress in key areas such as smart driving, smart industry, smart home, and smart city applications.

Figure 1.1 highlights the key end markets targeted by ST's solutions.



Figure 1.1: STMicroelectronics Activity Sectors

1.1.3 Global Presence

STMicroelectronics operates in more than 35 countries, with a strong presence in key markets around the world.

Figure 1.2 illustrates the company's global network, which includes [1]:

- **Manufacturing Sites:** ST has 14 main manufacturing sites, strategically located to serve its global customer base efficiently.
- **Sales & Marketing Offices:** With a network of sales and marketing offices, ST provides localized support and services to its customers.
- **R&D Centers:** ST's R&D centers are spread across the globe, fostering innovation and collaboration.

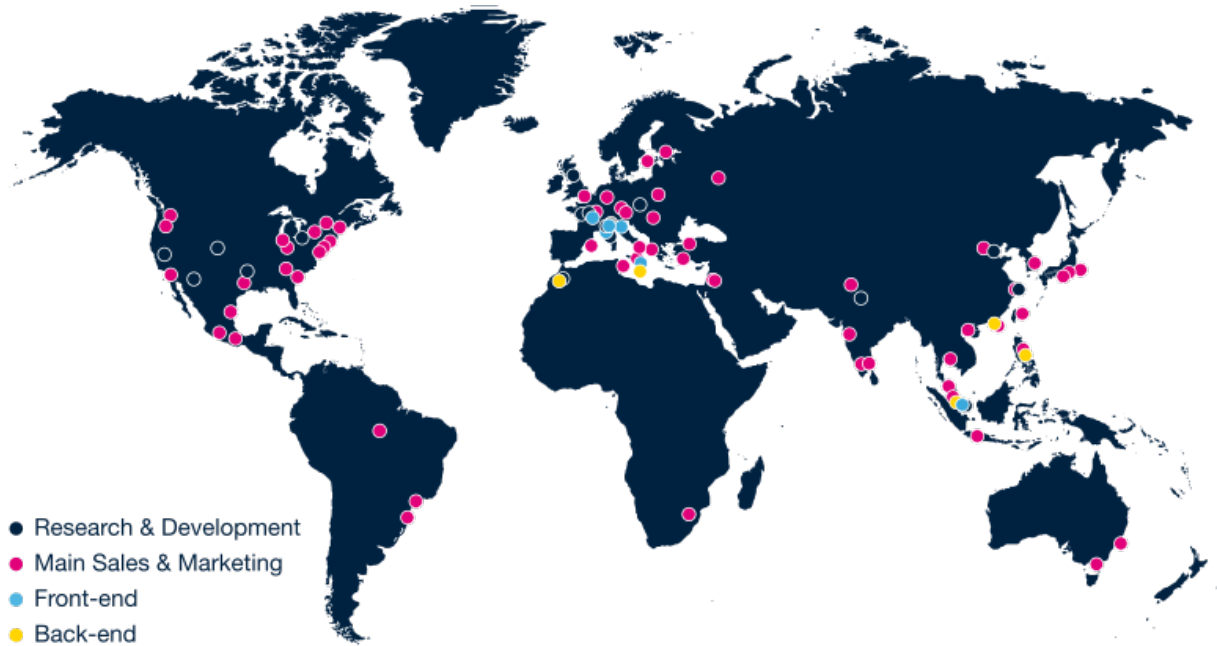


Figure 1.2: STMicroelectronics Worldwide Presence

1.1.4 ST Tunis

The STMicroelectronics Tunis site, located in the El Ghazela Technopark, employs over 100 professionals in various fields such as tools, applications, quality, and support functions. The facility is involved in all stages of the microelectronics industry, from initial circuit design to final verification before mass production.

With expertise in both hardware and software, the Tunis center's activities include electronic circuit design, software tool development, application software support, and the electrical validation and verification of new circuits. This diverse skill set allows the center to contribute significantly to ST's global operations. Figure 1.3 shows the site.

1.1.5 Application & Product Support Team

The Application & Product Support team is dedicated to delivering comprehensive customer support while managing and publishing STM32 product documentation. Their role includes providing in-depth technical support for ST's tools and products, helping customers troubleshoot problems and optimize their use of these



Figure 1.3: STMicronics Tunisia

technologies. They ensure the accuracy, clarity, and consistency of all technical documentation, which is regularly updated to reflect the latest developments and innovations.

The team is also involved in enhancing and maintaining existing products, as well as defining new products, derivatives, and application references to meet evolving market needs. Additionally, they lead initiatives related to intellectual property applications, offering specific support through the development of application notes, training programs, and other resources that help customers fully understand and utilize ST's products. This multifaceted approach ensures both customer satisfaction and continuous improvement in the STM32 ecosystem.

1.2 Project Context

1.2.1 Problem Statement

Debugging software offers a unified experience across many chips, vendors and architectures. It achieves this through a common layer of abstraction that presents heterogeneous targets behind a single conceptual model. The abstraction holds for simple primitives such as memory views and breakpoints. It fails for the advanced features, such as reconstructing execution flow, conditional and cross-triggering of debug events, or recovering execution history after a failure. Those features do not generalize, since each is tightly coupled to a vendor-specific implementation.

The gap between what a chip provides and what a developer can reach is the central concern of this project.

1.2.2 State of the Art

Debugging an embedded system from a host machine like a computer needs multiple components to work together, starting with the debug interface present on the microcontroller chip, the debugger software, and a hardware debug probe that links the microcontroller and the host machine.

Table 1.1: Embedded Systems IDE Vendors

Vendor	Toolchain	Debugger	Debug server	Debug probe
IAR	IAR toolchain (ICCARM)	C-SPY	Integrated	I-jet, I-jet Trace
Keil	Arm Compiler (armcc, armclang)	μ Vision debugger	Integrated	ULINK family
SEGGER	SEGGER toolchain	Ozone	J-Link GDB server	J-Link, J-Trace
STMicroelectronics	ST-Clang	GDB	ST-LINK GDB server	ST-LINK
Free	GCC, LLVM	GDB or LLDB	OpenOCD or pyOCD	CMSIS-DAP

The features the proprietary debuggers provide for devices based on the Arm architecture are worth examining in turn. ST's tools are specialized forks of open-source projects and are therefore excluded from this comparison.

Table 1.2: Debugger Features

Feature	IAR	Keil	SEGGER
ITM trace	✓	✓	✓ ¹
Instruction trace	✓	✓	✓
CTI support	✗	✗	✗
Scripting interface	✓	✓	✓
Multi-core debugging	Limited	Limited	Limited
IDE coupling	✓ ²	✓	✗

¹ Supports UART encoding only

² Implemented in the IDE, and able to work as a standalone server

1.2.3 Critique of Current State of the Art

Vendor IDEs expose a fixed set of tracing settings through their interface. they therefore reapplies the fixed configuration at every resume, which is consistent behavior from the point of view of the tool and leaves a developer no way to hold a configuration of their own.

Debug probes impose limitations of their own. IAR, Keil and SEGGER each supply two versions of their probe, a base model and an extended model that captures trace through dedicated pins.

Among the tools examined, none exposes on-chip trace capture through a probe other than its own, none decodes trace capture outside of a live debug session, or a capture obtained by other means than a debug probe.

Table 1.3: Debug Probe Limitations

Probe	Supported by IAR	Supported by Keil	Supported by SEGGER	Trace pin capture
I-jet	✓	✗	✗	✓ ¹
ULINK	✗	✓	✗	✓ ¹
J-Link, J-Trace	✓	✓	✓	✓ ¹
ST-LINK	✓	✓	✓	✗
CMSIS-DAP	✓	✓	✓	✗

¹ Given the appropriate model

² Established by evaluating the probes during this work, in the versions given in Table 1.2.

1.2.4 Proposed Solution

The proposed solution takes the form of four results.

An application note assembles into one document the procedure for using instruction trace on these devices.

An implementation of drivers that allow a host machine to configure the concerned trace and debug components.

A pipeline for trace decoding processing and analysis that is decoupled from the trace extraction means, and of a debugging session, allowing for offline trace analysis.

A debug engine carries that chain alongside the conventional debugging capabilities and presents the result inside a development tool during a live session.

1.3 Conclusion

This first chapter introduces the host organization, STMicroelectronics, and provides the context and objectives of the project. It sets the stage for understanding the foundational aspects of the project and its significance.

THEORY AND KEY CONCEPTS

Contents

1	Debugging Fundamentals	9
2	Debug Primitives	11
3	Classes of Debugging	12
4	Program Representation	13
5	Microcontrollers	15
6	The Arm Cortex-M Architecture	16
7	The Arm Debug Architecture	18
8	The CoreSight Trace Architecture	22
9	Debug and Trace Infrastructure in STM32 Microcontrollers	28
10	Conclusion	30

Introduction

This chapter establishes the concepts on which the remainder of this report depends. The presentation moves from the general to the particular. It opens with the definition of debugging and of the tool that performs it, then describes the operations a debugger provides and the categories into which debugging techniques fall. It continues with the representation of a compiled program, then narrows to microcontrollers, to the Arm M-profile architecture, to the debug and trace infrastructure that Arm defines for it, and finally to the implementation of that infrastructure in STM32 devices.

2.1 Debugging Fundamentals

2.1.1 Bugs, Specifications and Defects

A defect is a condition in a program that causes its behavior to violate its specification. Debugging is the process of investigating program behavior to identify the cause of such defects. The term *bug* is used interchangeably with defect.

Identifying a defect requires a specification to compare against. A program required to toggle an LED once per second provides a simple example. An LED that remains static, or that toggles at a different rate, is a case of a program violating its specification.

A specification states an end result, and that result implies intermediate behavior. Toggling an LED on a microcontroller requires initializing the device, configuring its power supply and clock tree, and writing to the registers that drive the output pin. A fault at any of these stages produces the same observable symptom, namely an LED that does not blink at the required rate. Writing to an incorrect register and omitting the clock configuration are distinct defects with identical symptoms.

Locating a defect requires identifying the stage at which the program departs from its specification. Debugging therefore depends on a means of observing the state of the system, either by executing the program under a debugger or by instrumenting the program to report it through logging.

2.1.2 Anatomy of a Debugger

A debugger is a software system that provides controlled interactions with a target system generally referred to as a *debuggee* for the purpose of examining and modifying its state.

Providing these operations requires two broad classes of work. The first is interpreting the representation of the program, which covers the machine code and the debug information held in the executable image. The second is interacting with the target.

A further distinction concerns presentation. The *frontend* is the program that provides the user interface

through which debug operations are requested and the state of the debuggee is displayed. The *backend* is the part that carries out the two classes of work mentioned above and maintains the state of a debugging session. For embedded systems such as a microcontroller, a hardware component named *debug probe* or *hardware probe* is generally needed to connect the host machine and the target debuggee. This element represents an intermediary communication channel between both systems. Figure 2.1 shows the decomposition.

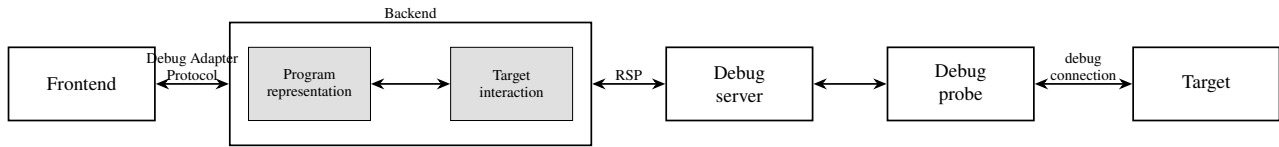


Figure 2.1: Conceptual decomposition of a debugger

Each junction in this chain is governed by a protocol. The connection between the probe and the target depends on the architecture of the device and is the subject of Section 2.7. The decomposition given here is conceptual, and a given system may distribute the responsibilities differently.

2.1.3 The Debug Adapter Protocol

The Debug Adapter Protocol defines the messages exchanged between a development tool and a *debug adapter*, a component that translates those messages into operations on the debugging mechanism beneath it [5]. A debugger is therefore not required to implement the protocol itself, since the adapter stands between the protocol and the interface the debugger already exposes [5].

The protocol describes the entities the developer works with, namely source files, line numbers, variables, threads and stack frames. It further permits requests and events outside the standard set, which allows an adapter to expose capabilities that the protocol does not describe.

2.1.4 The GDB Remote Serial Protocol

The GDB RSP is a point-to-point text-based communication protocol defined as part of the GNU debugger [6], that allows a debugger to connect to a remote target.

The debugger is the client and the target is the server. Every exchange is initiated by the client, and every transaction must be completed before initiating the next one[6].

The operations the protocol carries include reading and writing the processor registers, reading and writing memory, inserting and removing breakpoints, resuming execution and executing a single instruction [6].

A debugger may extend the set of packets. LLDB for example, defines packets of its own alongside the standard set [7].

2.2 Debug Primitives

In this work, *debug primitives* refer to the fundamental operations that a debugger exposes. Each is defined here in general terms.

2.2.1 Run Control

Run control provides mechanisms for starting, stopping and controlling the execution of the debuggee. Halting suspends execution and leaves the state of the program unchanged. Resuming returns the program to normal execution from the point at which it was suspended.

Stepping executes a limited portion of the program and then halts again. The size of that portion depends on the granularity requested, mainly, a single instruction or one source statement, which usually amounts to several machine instructions [10]. Further granularities are common, such as advancing over a complete function call rather than entering it.

2.2.2 Breakpoints: Software and Hardware

A breakpoint is an artificial stopping point inserted into a program for the purpose of debugging. When execution reaches the breakpoint, the program is suspended, after which execution may continue from the point at which it was suspended [10].

The mechanisms by which breakpoints are implemented are divided into two categories:

A *software breakpoint* replaces the instruction at the target address with a dedicated instruction that raises a debug event, and restores the original instruction once the breakpoint is removed.

A *hardware breakpoint* instead configures dedicated hardware components on the device such as comparators that watch the address of instructions fetched and raises a debug event on a match without modifying the program. Figure 2.2 contrasts the two mechanisms.

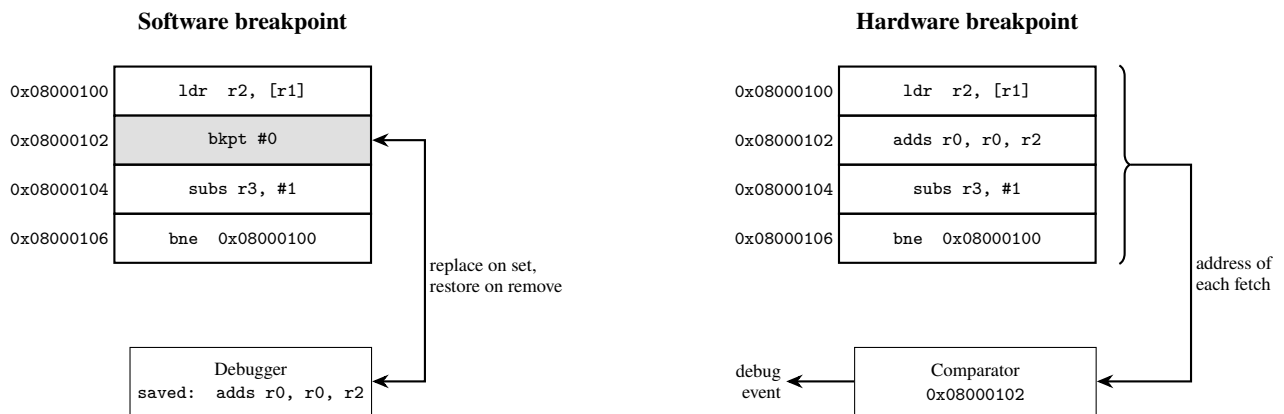


Figure 2.2: Software and hardware breakpoint mechanisms

2.2.3 State Inspection: Registers, Memory and Variables

The state of a program is observed through two elementary operations, which are reading the processor registers and reading memory [10].

These two operations are independent of the program, as neither carries the name, the type or the extent of the variable present in the source code. Presenting a variable therefore requires three further pieces of information, and is generally supplied by the debug information described in Section 2.4.3.

2.2.4 Call Stacks and Unwinding

The call stack is the sequence of subroutine activations that are in progress. An activation consists of a code location within the subroutine, an area of memory allocated on the stack termed a call frame, and the set of registers in use by the subroutine at that location [11].

Only the innermost activation is directly visible, since the processor registers describe that activation alone. Reaching the others requires *unwinding*, which proceeds one frame at a time. Starting from the current activation, the debugger restores the registers that this activation preserved and computes the address of the preceding call frame together with the corresponding code location[11].

2.2.5 Disassembly

Disassembly is the translation of the bytes held in the executable memory of the target back into machine instructions.

Disassembly serves two purposes in a debugger. It allows a developer to inspect code for which no source is available, and it allows the machine instructions produced by the compiler to be compared against the source statement they were generated from.

2.3 Classes of Debugging

Debugging techniques are grouped along several independent axes, and a given technique may occupy a position on each of them.

2.3.1 Intrusive and Non-Intrusive Debugging

A debug technique is termed intrusive, or invasive, when applying it alters the behavior of the program under examination. It is termed non-intrusive when it does not. Halting the processor and executing debug code through dedicated exceptions are presented as invasive mechanisms, whereas program trace and profiling are presented as non-invasive[8].

A program that meets a timing requirement when running freely may fail that requirement when it is

halted at a breakpoint, and a defect that depends on the relative timing of an interrupt and the main program may cease to appear once the timing is disturbed. Adding logging statements has a comparable effect.

2.3.2 Live and Post-Mortem Debugging

Live debugging examines a program while it is running or halted under the control of a debugger. The developer forms a hypothesis, sets a breakpoint, inspects state and repeats. The technique requires the defect to be reproducible while the debugger is attached.

Post-mortem debugging examines a record produced during execution and analyzed once that execution has finished. The term carries no implication that the program failed, since the record may equally describe a run that completed normally. The record may be a set of fault status registers, a region of memory preserved across a fault, a log, or a buffer of captured trace. No interaction with the program is possible during the analysis, so the analysis is limited to what the record contains. The technique applies where live debugging does not, for instance where a failure appears rarely, or where attaching a debugger would itself prevent it from occurring.

2.3.3 Run-Control and Trace-Based Debugging

A debugger that supports the operations of Section 2.2 is termed a run-stop debugger, or a run-time control debugger[10]. The developer directs the investigation by deciding where to stop and what to examine.

Tracing follows the opposite order. Tracing records the program flow and the data accesses of the program while it runs, and the recorded information is used at a later stage to perform run-stop debugging [10].

The two approaches answer different questions. A run-stop debugger answers what the state is at a chosen point. Tracing answers what the program did over an interval, including the path taken to reach a point of failure.

2.4 Program Representation

Section ?? established that source-level debugging requires a mapping between the source code and the machine code produced from it. This section describes how a compiled program is stored, what that mapping contains, and how machine code is translated back into instructions.

2.4.1 The Embedded Build Model

A program written in a high-level language passes through several tools before it can execute. The compiler translates each source file into an object file containing machine code together with unresolved references to entities defined elsewhere. The linker combines the object files, resolves those references, assigns an address to every function and variable, producing an executable image.

Embedded development has different properties in this aspect to host machines, mainly the fact that

addresses are fixed at link time and describe a physical device.

The debugger consumes the same image. extracting machine code to be disassembled, the addresses at which functions and variables reside, and the debug information relating those addresses to the source.

2.4.2 The ELF Format

The Executable and Linkable Format (ELF) is the object file format used by the toolchains considered in this work. Arm defines a processor-specific supplement to it for its own architecture [13].

2.4.2.1 Segments, Sections and the Loadable Image

An ELF file describes its contents in two manners. The section header table describes the file as a set of *sections*, which groups related content under a name and a type, so that machine code, initialized data, uninitialized data and debug information each occupy sections of their own. The program header table describes the file as a set of *segments*, which is the view used when the program is loaded. A segment of loadable type identifies a range of the file together with the address at which that range must appear in memory.

2.4.2.2 Symbol Tables

A symbol table is a data structure where each symbol in the source code is associated with the necessary information for its functioning. Each entry records the name of an entity, its address, the number of bytes it occupies, and its kind, which distinguishes a function from a data object among other categories.

2.4.2.3 Arm Mapping and Tagging Symbols

A section of an ARM ELF file can contain a mixture of ARM code, Thumb code and data. The mapping and tagging symbols are used in the symbols table to denote the start of a specific section of the appropriate type.

Table 2.1: Arm Mapping Symbols

Symbol	Definition
\$a	Denotes the start of a region of code containing ARM instructions.
\$t	Denotes the start of a region of code containing Thumb instructions.
\$d	Denotes the start of a region of data.

2.4.3 DWARF Debug Information

DWARF is the debug format used by the toolchains considered here. It describes a program as a tree in which nodes represent functions, data and types, provides a mapping between executable instructions and the corresponding source code lines, and describes how to unwind the stack [10]. It is stored in dedicated sections of the ELF file. Table 2.2 states the constructs it defines and the debugger operation each supports.

Table 2.2: DWARF constructs and the operations they support

Construct	What it describes	Operation it supports
Debugging information entries	The program as a tree of functions, data and types, nested as the source is nested.	Determining which names are visible at a given point.
Line number program	The correspondence between instruction addresses and source lines.	Presenting a location in source terms and stepping from line to line.
Inlined subroutines	The expansion of a subroutine at its call site, and the chain of source locations that results.	Reporting the call site rather than the expanded code alone.
Type descriptions	How the bytes belonging to a variable are to be interpreted.	Displaying a variable rather than a sequence of bytes.
Location descriptions	Where an object resides, which may change over its lifetime.	Finding a variable at the current point of execution.
Call frame information	Where registers are saved and how the address of the preceding frame is computed.	Unwinding the call stack.

2.5 Microcontrollers

A microcontroller is a system that integrates a processor, its memories and a set of peripherals within a single component. It cannot host a debugger, since it has neither the memory to hold one nor the means for a developer to operate one on it [10].

2.5.1 Device Description: CMSIS-SVD

The CMSIS System View Description format formalizes the description of the system contained in Arm Cortex processor-based devices, in particular, the memory mapped registers of peripherals. The detail contained in system view descriptions is comparable to the data in device reference manuals. The information ranges from high level functional descriptions of a peripheral all the way down to the definition and purpose of an individual bit field in a memory mapped register. [14].

The format is based on XML and describes a single device per file. A device consists of a processor and at least one peripheral, each peripheral contains at least one register, and a register may consist of one or more fields [14]. Each level carries attributes, such as the address of a register, its reset value, the width and position of each field, whether a field may be read or written, and the meaning of particular field values.[14].

2.6 The Arm Cortex-M Architecture

This section describes the parts of the architecture that the debug and trace infrastructure of Sections 2.7 and 2.8 depends upon.

2.6.1 Arm Architecture

Arm divides its architecture into three profiles, addressed to different classes of application. The A-profile targets systems running general-purpose operating systems, the R-profile targets systems with real-time requirements, and the M-profile targets microcontrollers.

2.6.2 Register Model and the Thumb Instruction Set

The M-profile presents thirteen general-purpose registers, a stack pointer, a link register, a program counter, and a program status register carrying the condition flags and the current execution state.

The M-profile executes the Thumb instruction set only.

2.6.3 Exception Model and Vector Table

In Arm architectures, an exception is defined as any events that suspends the currently executing program, this can originate from a peripheral signal termed an interrupt, a fault, or a software request.

An exception requires a handler, which is a subroutine that is executed whenever the exception occurs. And the address of the entirety of the handlers are stored contiguously in a table termed the Vector Table, which is then indexed by the number of the exception to retrieve the appropriate handler.

2.6.4 System Address Map

The M-profile defines a default address map that assigns a purpose to each region of the address space. Distinct regions are reserved for code, for on-chip static memory, for peripherals, and for external memory and devices [8], [9].

A further region is reserved for the Private Peripheral Bus (PPB), which holds the registers of the system and debug components belonging to the processor [8], [9].

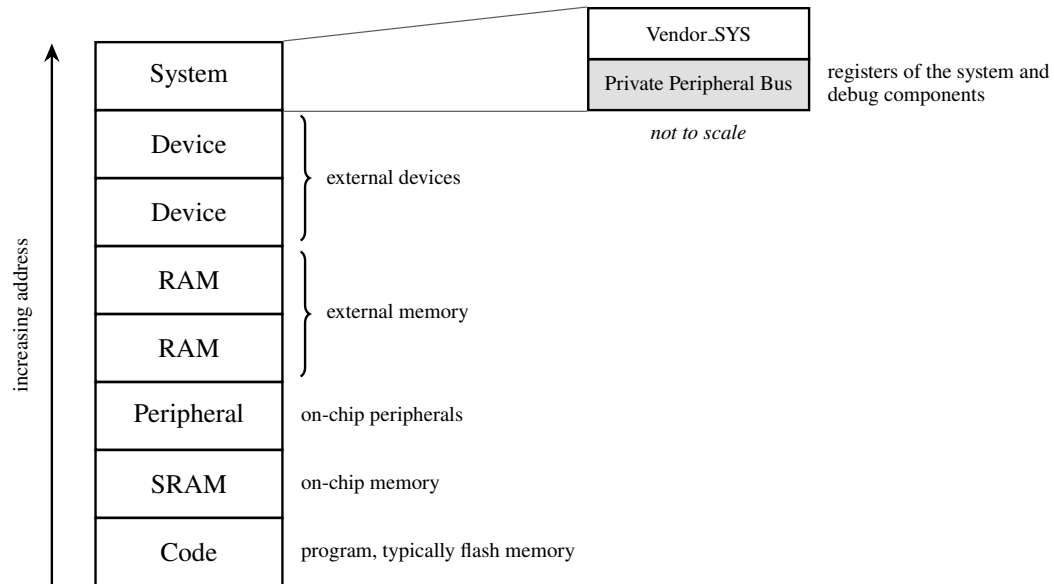


Figure 2.3: The M-profile system address map [8]

2.6.5 AMBA Buses and their Roles in Debug

The components of an Arm device are connected by buses defined in the Advanced Microcontroller Bus Architecture (AMBA). Table 2.3 states the four most common ones.

Table 2.3: AMBA buses and their roles in debug and trace

Bus	Name	Role
APB	Advanced Peripheral Bus	Carries configuration accesses to debug components. Transactions arriving from a debug connection are converted into APB transactions and transported to each of the components the debugger must access [15].
AHB	Advanced High-performance Bus	The protocol through which the debug registers of a Cortex-M processor are commonly reached. Accesses may be directed to all locations in the memory space of the processor rather than to the debug registers alone [15].
AXI	Advanced eXtensible Interface	Carries accesses into the main functional memory space of the devices that implement it, bypassing the caches and the memory map controls of the processor [15]. Where a device routes captured trace into system memory, this is also the interface over which that trace is written, as Section 2.8.6 describes.
ATB	Advanced Trace Bus	Reserved for trace. It is a common bus used by the trace components to pass trace data in a data-agnostic format [16], which is what allows one infrastructure to transport several kinds of trace.

2.7 The Arm Debug Architecture

Section 2.1.2 left the junction between the probe and the target undescribed. This section describes the specification that governs it and the debug logic it reaches.

2.7.1 The Arm Debug Interface

The Arm Debug Interface (ADI) defines the means by which a debugger accesses the debug functionality provided by debug components in an embedded system on chip [17].

An implementation of that specification within a device is termed a Debug Access Port (DAP), which provides a debugger with a standard interface for accessing debug resources in systems that expose their debug information by resource-specific methods [17].

The Debug Access Port presents one interface to the debugger and performs the resource-specific access on its behalf. It is composed of two kinds of elements, a Debug Port and one or more Access Ports, Figure 2.4

shows an example configuration of a DAP.

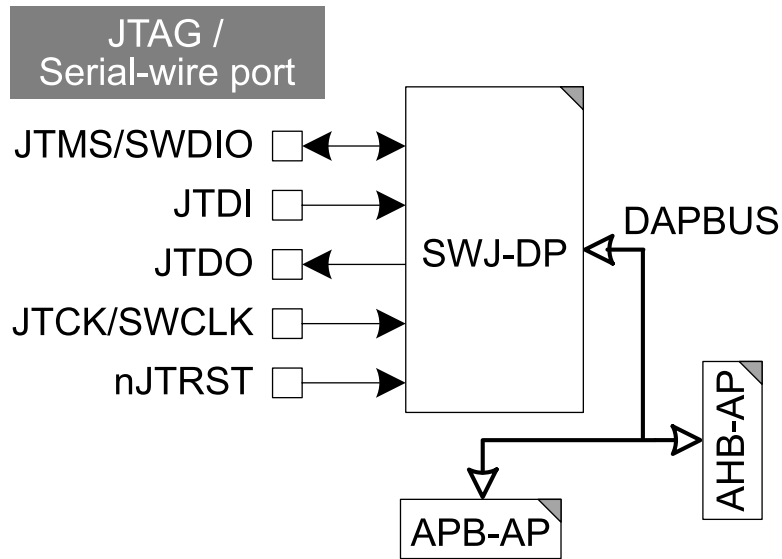


Figure 2.4: Structure of a Debug Access Port[18]

2.7.2 Debug Port: JTAG and SWD

A Debug Port provides four things[17]:

- The external physical connection to the interface.
- A method of obtaining the identification code of the Debug Access Port.
- The methods by which the debugger accesses the Debug Port and Access Port registers.
- A method of aborting a register access that appears to have failed.

There are three types of Debug Ports,

- JTAG Debug Port is accessed by scan chains compliant with IEEE 1149.1.
- Serial Wire Debug Port (SWD) is a two-pin serial interface using a packet-based protocol to read and write registers.
- SWJ Debug Port which combines the two above[17].

The debugger drives serial accesses that scan values into registers inside the Debug Port, which are then converted into bus transactions directed at an Access Port register [15].

2.7.3 Access Ports and the Memory Access Port

An Access Port uses a resource-specific transport mechanism to access debug information in the system, and passes that information to the Debug Port [17]. A Debug Access Port contains at least one Access Port and may

contain several [17], each driving a different memory space, with the type of Access Port determined by the bus protocol required for that space [15].

2.7.4 Component Identification and ROM Tables

CoreSight designates two mechanisms that allow a debugger to discover the debugging capabilities of an unfamiliar device.

The first is a common register layout. A component compliant with the CoreSight architecture occupies a block with a size that is a multiple of four kilobytes, divided between a set of predefined management registers and a set of registers specific to the device [15].

The second is ROM tables. A ROM table always occupies four kilobytes and is part of the address space of the memory system connected to an AP [17]. Its entries point to the components present, and one bit of each entry indicates whether the corresponding component is implemented [8], [9].

2.7.5 Halting Debug Control

On an M-profile device, halting debug capabilities are controlled by the three memory-mapped registers described in table 2.4[8], [9].

Table 2.4: Registers governing halting debug

Register	Control	Effect
DHCSR	C_DEBUGEN	Enables halting debug, after which a debug event halts the processor in Debug state.
	C_HALT	Requests a halt when written as one, and a resume when written as zero. The processor also sets it on entry to Debug state.
	C_STEP	Requests a single step.
	C_MASKINTS	Prevents configurable interrupts from being taken during a step, which is what allows a step to remain within the statement under examination.
	S_HALT	Reports that the processor is in Debug state. Read-only.
DEMCR	MON_EN	Enables the DebugMonitor exception, the alternative to halting.
	Vector catch	Arms the mechanism described in Section 2.7.6.
	TRCENA	Governs the trace components.
DFSR	One bit per event	Reports why execution stopped, distinguishing an internal halt request, a breakpoint, a watchpoint match, a vector catch and an external debug request. A debugger reads it after a halt in order to report the cause to the developer. Each bit is cleared by writing one.

2.7.6 Reset Types and Vector Catch

M-profile architectures define two types of reset, A Local reset is one initiated through the Debug Access Port, and resets only the core and the debug logic, a System reset on the other hand resets the entire SoC. Vector Catch is a debug mechanism implemented in Cortex-M devices that allows the device to enter Debug State upon exception entry without requiring a breakpoint.

2.7.7 Breakpoint and Watchpoint Units

The Flash Patch and Breakpoint unit (FPB) is the debug component that implements hardware breakpoints. Its relevant capability is that it can set breakpoints on code held in read-only memory [8].

The Data Watchpoint and Trace unit (DWT) implements generic comparators that can be configured to provide various functionalities such as watchpoints, data tracing or signaling for external resources. Moreover,

these comparators can be configured to signal a match on either an instruction address, a data address or a data value given that the prior comparator is a data address one.

2.8 The CoreSight Trace Architecture

This section describes the architecture that Arm defines for trace. CoreSight is the mechanism Arm uses to provide the connectivity between a debugger and a processor, and it defines a set of capabilities that may be designed into a processor or into system-level components [15].

2.8.1 Trace Sources, Links and Sinks

Trace components fall into three categories. A trace source is a component that generates trace data. A trace sink is a component that stores or outputs trace data. A trace link is a component that connects trace components together [19]. Any trace path in a device is therefore a chain that begins at a source, passes through zero or more links, and terminates at a sink.

Since a design is free to implement any subset of these components, the topology differs from device to device. A debugger consequently needs to know the topology of the particular target before it can configure a trace path [19].

2.8.2 Instruction, Data and Instrumentation Trace

Trace refers to the process of capturing data that illustrates how the components in a design are operating, executing and performing, and the Arm approach usually involves a separate generation component for each type of trace [19]. Table 2.5 states the types that concern this work.

Table 2.5: Types of trace

Type	What it records	Intrusive
Instruction	The execution history of the processor, and the places where execution behaves unexpectedly [19]. This is the type on which this project is based.	✗
Data	Each data access, together with the address and the value transferred [19]. The trace architecture leaves it optional [19].	✗
Instrumentation	Events and values supplied by the software itself, in the style of a print statement [19].	✓

Instrumentation trace differs from the other two in that the program must be modified to produce it,

which places it on the intrusive side of the distinction of Section 2.3.1, albeit at a low level of intrusion [8].

2.8.3 Trace Source: the Embedded Trace Macrocell

The component that generates instruction trace is termed a trace unit. It traces the instructions a processor executes and the data those instructions act upon [12]. The implementation Arm supplies for this purpose is the Embedded Trace Macrocell (ETM), which exists in four architecture versions [12], [20]. This work treats ETMv4.

2.8.3.1 Functions of a Trace Unit

A trace unit has two main functions. The first is to generate trace, by producing trace streams. The second is to filter those streams, so that tracing may be made active only for particular threads of execution or particular functions [12].

The second function follows from a property of the first. A processor executes on the order of hundreds of millions of instructions per second, whereas the path that carries the trace has a fixed bandwidth and the storage at its end a fixed size. Filtering lowers the rate at which trace is produced, so that the path and the storage can keep up.

2.8.3.2 Trace Streams and Trace Identifiers

A trace unit conforming to ETMv4 generates an instruction trace element stream and, where data tracing is implemented and enabled, a data trace element stream [12]. The two are independent, and the architecture requires neither that they be captured by the same device nor that their capture be ordered [12]. Interpreting the data stream, however, typically requires the instruction stream to be interpreted as well [12].

Several streams may share one bus. Each stream therefore carries its own identifier, and Section 2.8.4 describes how it travels with the data.

2.8.3.3 Trace Elements

A trace unit reports execution as a stream of elements [12].

The stream does not contain an element for every executed instruction. An element is generated when the processor executes a direct branch, an indirect branch or an instruction synchronization barrier, when it takes or returns from an exception, and when it enters Debug state [12]. A wait-for-interrupt or wait-for-event instruction generates one only where the unit reports that it does [12]. Each element therefore implies the execution of every instruction from the target of the previous element up to the instruction it reports [12]. A conditional branch needs one bit, namely whether it was taken, and the instructions between two branches need none, since the program image already holds them. An *atom* element carries that bit.

Addresses follow the same principle. A branch whose target is encoded in the instruction needs no address in the trace, since the target may be read from the program image. The trace unit therefore omits an address wherever it can be inferred from the program image and from what the stream has already reported, and transmits only those that cannot [12].

Table 2.6 states what each kind of element reports.

Table 2.6: Kinds of trace element

Element	What it reports
Atom	Whether a conditional branch was taken.
Address	A branch target that cannot be inferred from the program image.
Exception and exception return	Entry to an exception and return from it, which occur at an instruction boundary with no branch appearing in the code.
Synchronization	Where a packet starts, the configuration of the trace run, and the address from which to begin reading.
Overflow	A gap in the trace, and the reason for it.
Timestamp and cycle count	Values from a global counter and counts of processor clock cycles.

Synchronization elements are generated periodically, which matters because trace held in a circular buffer is overwritten from an arbitrary point. A captured trace is therefore not necessarily continuous, and an analysis must represent a gap explicitly rather than joining the two sides of it [12]. Figure 2.5 works the principle through for a loop of four iterations, in which three taken atoms and one not-taken atom stand for sixteen executed instructions.

Program		Trace elements		Reconstructed execution		
0x08000100	ldr.w r2, [r1], #4	Address	0x08000100	E	0x08000100 ldr.w r2, [r1], #4	
0x08000104	add r0, r2	Atom	E		0x08000104 add r0, r2	
0x08000106	subs r3, #1	Atom	E		0x08000106 subs r3, #1	
0x08000108	bne.n 0x08000100	Atom	E		0x08000108 bne.n 0x08000100	
		Atom	N	E	0x08000100 ldr.w r2, [r1], #4	
		E: branch taken N: not taken			0x08000104 add r0, r2	
					0x08000106 subs r3, #1	
					0x08000108 bne.n 0x08000100	
				E	0x08000100 ldr.w r2, [r1], #4	
					0x08000104 add r0, r2	
					0x08000106 subs r3, #1	
					0x08000108 bne.n 0x08000100	
				N	0x08000100 ldr.w r2, [r1], #4	
					0x08000104 add r0, r2	
					0x08000106 subs r3, #1	
					0x08000108 bne.n 0x08000100	

Figure 2.5: Trace elements generated for a short loop. Synchronization elements are omitted.

2.8.3.4 Packet Encoding

Elements are encoded into packets for transmission. A packet is a header byte followed by zero or more payload bytes, and the packet type determines how many [12]. A header is unique within a stream, though the same byte may mean one thing in the instruction stream and another in the data stream, so a decoder must know which stream it reads [12].

Elements and packets do not correspond one to one. One atom packet may carry several atom elements, and six atom packet formats exist for that purpose [12]. Decoding is therefore two operations, namely recovering packets from the byte stream and expanding them into elements.

The trace alone does not state which instructions executed, since it reports branches and omits the addresses it expects to be inferred. Reconstruction requires the program image [12]. The image must be the exact one the processor ran, as any difference between it and the executed code corrupts the decoded output [19].

2.8.4 The CoreSight Trace Formatter

Where several trace streams share one path, the identity of each must travel with its data. Formatters are the method by which trace source identifiers are wrapped into the output trace data stream, and the format specifies how trace sinks embed the source identifiers carried on the ATB into a single stream [21].

The protocol outputs data in frames of sixteen bytes, arranged as Figure 2.6 shows. The architecture specification gives the meaning of every byte in full [21].

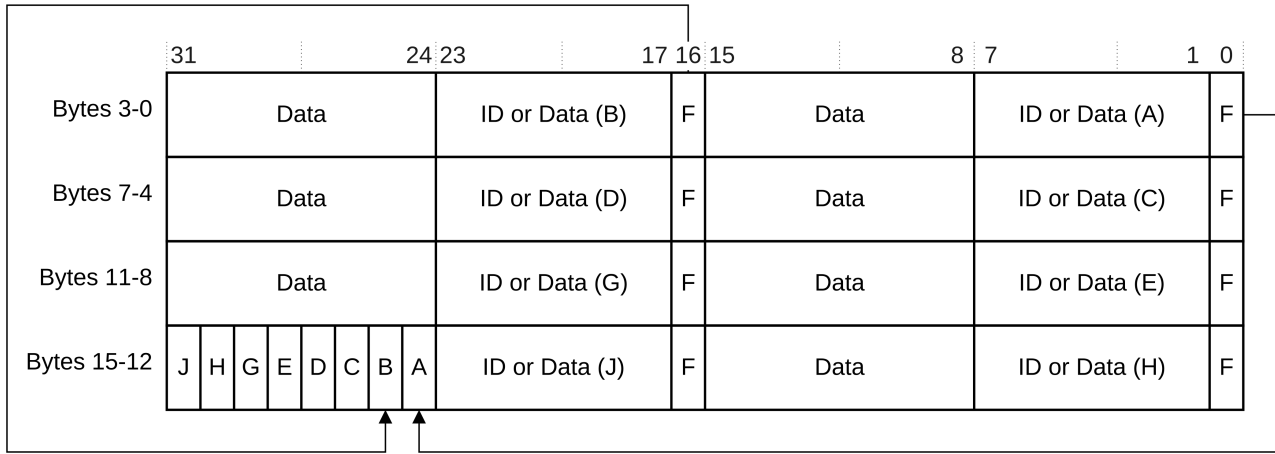


Figure 2.6: Structure of a formatter frame, taken from [21]

The design serves several ends at once [21]:

- Trace from several sources may be merged into a single stream and later separated.
- No requirement is placed on the data emitted by the sources.
- The result may be stored as a bitstream without separate alignment information.
- The result may be decoded even where the start of the trace has been lost.

Where only a single trace source is used and the embedded trigger and flow control information is not required, formatting may be disabled to obtain better throughput [21]. A decoder must therefore be told whether the stream it receives is formatted, since a formatted stream requires the frames to be unwrapped before the packet decoding of Section 2.8.3.4 may begin.

2.8.5 Off-Chip Trace Sinks: TPIU and SWO

Two families of sink exist, distinguished by where the trace data comes to rest. The first exports it from the device.

The standard component for this purpose is the Trace Port Interface Unit (TPIU), which drives the external pins of a trace port so that capture hardware may receive the data [22]. It also inserts source identifiers into the stream and coordinates the stopping of capture when a trigger is received [22].

The limitation of this route is bandwidth. A trace port is a small number of pins running at a fraction of the internal frequency, so it carries far less than the on-chip trace bus and suits a low rate of trace over a long period [15]. The Serial Wire Output (SWO) passes trace over a single pin directly to a debugger, at a lower rate still [19].

Two practical constraints follow. Pins must be available on the package and routed on the board, and the capture equipment must be able to receive them, which usually means a probe more capable than the one

used for ordinary debugging. Neither constraint is met by every board.

2.8.6 On-Chip Trace Sinks: the Trace Memory Controller

The second stores it inside the device, which is what the Trace Memory Controller (TMC) does [19].

Two axes describe the component. The configuration is fixed when the design is integrated into a system, and Table 2.7 states the three that exist. The operating mode is chosen when the component is programmed, and Table 2.8 states the three that exist [23].

Table 2.7: Trace Memory Controller configurations

Configuration	Storage	Retrieval	Memory size range
ETB	Dedicated SRAM	Debug bus	512 bytes to four gigabytes
ETF	Dedicated SRAM	Debug bus, or drained over the trace bus	512 bytes to four gigabytes
ETR	System memory, over AXI	Debug bus	The size of the memory region

Table 2.8: Trace Memory Controller operating modes

Mode	Description	Configurations
Circular buffer	The storage is used as a circular buffer. Old trace is overwritten once it is full, and nothing may be read until capture has stopped.	ETB, ETF, ETR
Hardware FIFO	The component acts as a link between a source and a sink, applying back pressure rather than losing trace.	ETF
Software FIFO	The component acts as a queue that a debugger reads over the debug bus while capture continues.	ETB, ETF, ETR

The dedicated memory of the ETB and ETF configurations is normally small, so triggering and filtering are usually necessary to ensure that important trace is not lost through wrapping [19].

Table 2.9 sets the two routes against each other.

Table 2.9: On-chip and off-chip trace capture

	On-chip	Off-chip
Bandwidth	The full internal bus rate	A fraction of the internal rate
Size	The size of the memory	The size of the capture device
Throughput	Read out after capture	Continuous during capture
Extra hardware	None	Trace pins, board routing and a capture probe

2.9 Debug and Trace Infrastructure in STM32 Microcontrollers

The preceding two sections described what Arm specifies. A device manufacturer decides which of those components to implement, where to place them, and what to add. This section describes the choices that are common across the STM32 range and the respects in which its members differ.

2.9.1 Debug Access and Trace Capabilities

Every STM32 device exposes a debug port supporting SWD, and the devices built on the larger cores support JTAG as well [24]. Behind that port lie the access ports of Section 2.7. The architecture fixes the address of every component it defines. The manufacturer chooses the address of every component it adds, and the reference manual for the family states them. A debugger therefore knows part of the topology in advance and must either be told the rest or read it from the ROM tables of Section 2.7.

Debug capability varies across the range, since it follows in part from the processor a given family implements. Table 2.10 states which families implement a trace unit and which additionally implement an on-chip trace sink.

Table 2.10: Trace capability of the STM32 families surveyed

Family	Processor	Instruction trace	Trace unit architecture	On-chip trace
C0, F0, G0, L0, U0	Cortex-M0 and M0+	✗	–	✗
F1, F2	Cortex-M3	✓	ETMv3	✗
F3, F4, G4, L4, L4+	Cortex-M4	✓	ETMv3	✗
F7	Cortex-M7	✓	ETMv4	✗
H7, H7RS	Cortex-M7	✓	ETMv4	✓
C5, H5, L5, U3, U5	Cortex-M33	✓	ETMv4	✗
N6	Cortex-M55	✓	ETMv4	✓

The survey covers the mainstream, high-performance and ultra-low-power families. It follows AN4989 [24] where that document covers the family, and the reference manual of the family otherwise, of which RM0433 [25] and RM0477 [18] describe the two devices carrying an on-chip sink that this work uses.

The last two columns are read together. Every family carrying an on-chip trace sink implements ETMv4, so an implementation that supports the later architecture alone loses no device reachable by the on-chip route of Section 2.8.6. The converse does not hold, since several families implement ETMv4 and carry no sink, and on those the trace must leave through the pins.

2.9.2 The DBGMCU Component

STM32 devices add a component that Arm does not define, named the microcontroller debug unit (DBGMCU). Unlike the units of Section 2.7, it is placed in the ordinary peripheral address space rather than in the PPB. It reports the identity of the device, which a debugger reads in order to determine which memory map and which peripheral description to apply, and it carries the controls described below. Its address and the exact layout of its registers are properties of the family and are stated in the reference manual.

2.9.2.1 Peripheral Freeze

Halting the processor does not halt the peripherals, so a peripheral continues to operate while the program that services it is suspended.

STM32 devices provide a set of freeze registers in the DBGMCU component, one register per peripheral bus. Each bit corresponds to one peripheral and stops that peripheral while the processor is halted for debug [24]. The registers may be written either by the debugger or by the application [24].

2.9.2.2 Low-Power Mode Debug Support

The processor is not clocked in the low-power modes of the device, so a debug connection is lost when the application enters one of them [24].

The configuration register of the DBGMCU component carries one control for each low-power mode. Setting it keeps the clocks required for debug running while the device is in that mode, which retains the session [24].

2.9.2.3 Trace Pin and Clock Control

The DBGMCU component also carries the controls that make trace usable. It enables the trace port clock [18]. On the families that route trace to pins, it also assigns those pins [26]. On devices divided into power domains, it enables the debug clock of each domain separately [25]. Which of these are present is a property of the family.

2.10 Conclusion

This chapter has assembled the concepts on which the remainder of the report rests. It defined a defect in terms of a specification, and a debugger as the system that examines and controls a program in order to locate one. It described the operations such a system provides and the protocols that connect its parts. It then set out the representation of a compiled program, comprising the object file that holds the image, the debug information that relates that image to the source, and the decoding that recovers instructions from the bytes. It narrowed to microcontrollers and to the Arm M-profile, described the ADI through which a debugger reaches a target, and described the CoreSight architecture through which a device reports what its processor executed.

Two results of this chapter are used repeatedly in what follows, and they belong to the two halves of it. The trace infrastructure of Section 2.8 produces a record of execution expressed in addresses, and produces it in a form that cannot be interpreted without the program image. The debug information of Section 2.4 converts an address into a function, a source file and a line. Neither is sufficient alone. Combining them is what turns a buffer of trace bytes into an account of a program in the terms in which it was written, and Chapters 4 and 5 describe how this combination was achieved.

OBJECTIVES SPECIFICATION AND WORK ENVIRONMENT

Contents

1	Project Objectives and Deliverables	32
2	Requirements Analysis	33
3	Hardware Environment	34
4	Software Environment	36
5	Conclusion	37

Introduction

Chapter 1 identified a gap between the debug and trace capability a microcontroller implements and the capability a developer can reach through available tools. This chapter states what the project undertook to do about that gap, the requirements derived from it, and the hardware and software used to produce the result.

3.1 Project Objectives and Deliverables

The internship produced the three deliverables of Table 3.1. Chapter 5 presents the chronology.

Table 3.1: Deliverables

Deliverable	Description
Application note	The instruction trace infrastructure of STM32 microcontrollers, a reference implementation for decoding the trace such an infrastructure produces, and guidance on working with instruction trace features on vendor IDEs.
Proof of Concept	A demonstration that instruction trace is obtainable on an STM32 device using an on-chip trace sink.
Debugger	A debugger with instruction trace features.

3.1.1 Scope

Table 3.2: Scope of the three deliverables

Deliverable	Scope
Application note	Instruction trace infrastructure of every STM32 family that implements it
	Keil μ Vision with the ULINKpro probe
	IAR Embedded Workbench with the I-jet Trace probe
Proof of Concept	ETMv4 implementations
	On-chip trace sink
	Instruction trace
Debugger	STM32 MCUs
	Breakpoints and variable inspection
	Peripheral inspection, memory views and disassembly
	Instruction trace

3.2 Requirements Analysis

3.2.1 Functional Requirements

Table 3.3: Functional requirements

ID	Requirement
FR1	Configure the trace unit and the on-chip trace sink of the target through the debug connection.
FR2	Start and stop trace capture in step with the run state of the program, without a separate command from the developer.
FR3	Extract the captured trace over the same debug connection that carries every other debug operation.
FR4	Decode the extracted byte stream into trace elements.
FR5	Reconstruct the sequence of executed instructions from those elements together with the program image.
FR6	Attribute each reconstructed instruction to a function, a source file and a line.
FR7	Represent a discontinuity in the capture explicitly, rather than joining the two sides of it.
FR8	Present the reconstruction inside a development tool during a live debug session.
FR9	Provide the run control primitives of Section 2.2, namely halting, resuming, stepping, breakpoints and watchpoints.
FR10	Read and write memory and processor registers.
FR11	Present the registers of a peripheral by name and by field, using a description of the device.
FR12	Provide disassembly and source-level inspection of variables.
FR13	Reset the target and restore the debug session to a state consistent with the reset device.

3.2.2 Non-Functional Requirements

Table 3.4: Non-functional requirements

ID	Requirement
NFR1	Require no hardware beyond the debug probe already present on the development board, and no trace pins.
NFR2	Operate on Windows and on Linux.
NFR3	Integrate with existing development tools.

3.2.3 Traceability to Deliverables

Table 3.5: Traceability of requirements to deliverables

Deliverable	Functional	Non-functional
Application note	FR1 to FR5	NFR1
Proof of Concept	FR1 to FR8	NFR1
Debugger	FR8 to FR13	NFR2, NFR3

3.3 Hardware Environment

3.3.1 Microcontroller: STM32H7S3L8

The main device used during the development period of the project is an STM32H7S3L8, a member of the STM32H7RS family. It carries an Arm Cortex-M7 processor [27] running at up to 600 MHz with a memory protection unit, a double-precision floating-point unit and separate instruction and data caches of 32 kilobytes each. The device provides 64 kilobytes of internal flash memory and 620 kilobytes of internal SRAM, and is supplied in a TFBGA225 package [28].

Figure 3.1 shows the debug and trace components of the device and the buses that connect them.

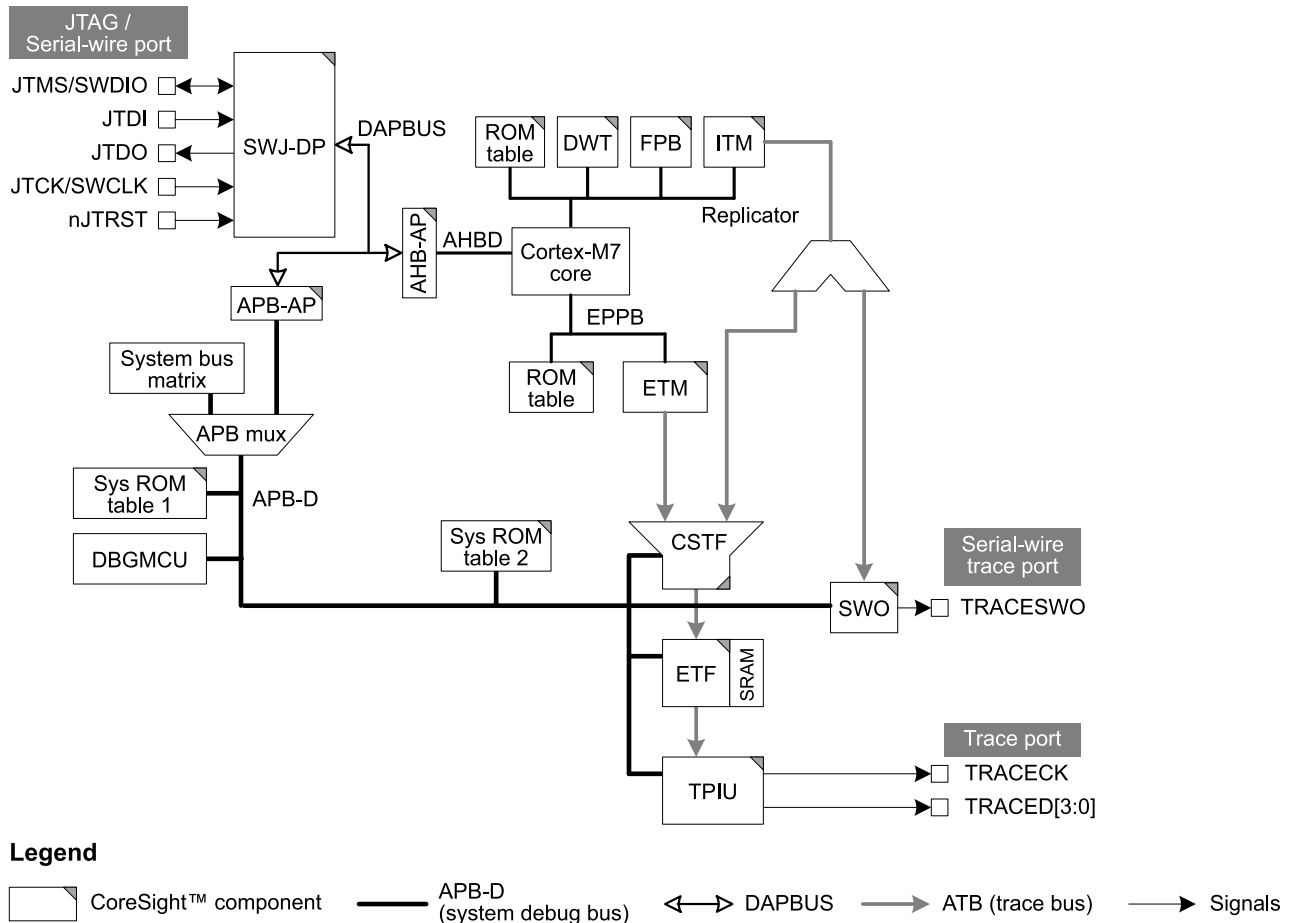


Figure 3.1: Debug and trace infrastructure of the target, adapted from [18]

3.3.2 Development Board: NUCLEO-H7S3L8

The microcontroller is used on a NUCLEO-H7S3L8. It exposes a MIPI20 connector for external debug tools, which carries the serial wire and JTAG signals together with a four-bit trace port [29].

3.3.3 Debug Probe: STLINK-V3EC

The probe is the STLINK-V3EC embedded on the board, which supports both SWD and JTAG [29]. It connects to the host through a USB Type-C connector and additionally provides a virtual COM port (VCP) and mass storage programming.

3.3.4 Trace Resources Available on the Target

Table 3.6: Trace components of the target device

Name	Base address	Note
ETM	0xE0041000	Implementation of the Cortex-M7 ETM, configured for instruction trace only [30].
TMC	0xE00F4000	ETF configuration, providing two kilobytes of memory.
TPIU	0xE00F5000	-
Funnel	0xE00F3000	Funnels the ETM trace data into the TMC. It also carries the stream of the instrumentation macrocell [18], which this work does not use.

3.4 Software Environment

3.4.1 Development Environment

Table 3.7: Development environment

Element	Choice
Host platforms	Linux and Windows
Backend language	C++23
Frontend language	TypeScript
Compiler	Clang
Build system	CMake, Ninja generator
Frontend bundler	esbuild
Editor	Visual Studio Code

3.4.2 External Software Components

Table 3.8: External software components

Component	Version	Description	Justification
OpenOCD	Fork of 0.12.0	Open-source on-chip debugger [31].	Supplies the groundwork the trace drivers extend.
LLVM	22	Compiler infrastructure and debugger, of which the machine code layer [32] and the debugger [33] are used.	Supplies the core debug functionality and the analysis of the program image.
OpenCSD	1.8.3	Library for decoding CoreSight trace [34].	Simplifies the trace decoding process.
Xerces-C	3.2.4	XML parser.	Reads the CMSIS-SVD device descriptions.
CodeSynthesis XSD	4.2.0	Generator of C++ bindings from an XML schema.	Avoids writing the CMSIS-SVD bindings by hand.
Visual Studio Code	1.125	Editor hosting the frontend extension.	Already implements the client side of the Debug Adapter Protocol.

3.5 Conclusion

This chapter has stated the three deliverables of the project, the requirements derived from them, and the environment in which they were produced. Two facts from it govern everything that follows.

The first is the hardware. The target implements both routes by which trace leaves a processor, and the bench can use only one of them, since the probe available cannot capture a trace port. The on-chip sink is therefore the route the work takes, and its capacity of two kilobytes is the constraint against which the results are read.

The second is the requirement set. The functional requirements describe a pipeline running from the configuration of a trace unit to the display of an executed source line, and the non-functional requirements constrain it to hardware already present and to an existing development tool. Chapter 4 describes the structure built to satisfy them.

ARCHITECTURE

Contents

1	System Overview	39
2	Instruction Trace Architecture	40
3	Debugger Architecture	42
4	System-Wide Rules	46
5	Conclusion	47

Introduction

This chapter states the structure of the solution. It names the components, states what each is responsible for, and states why the responsibilities were divided as they were. Implementation is the subject of Chapter 5.

4.1 System Overview

4.1.1 Global Architecture

The frontend presents the debug session to the developer. The engine implements the debug logic. The debug probe is reached from the host over USB and reaches the target over SWD. The target executes the program under examination.

The frontend and the engine are separate programs joined by the Debug Adapter Protocol. That protocol is the only boundary in the system between separate programs. Every decision about what a debug operation means is taken inside the engine.

The engine contains a debug server, linked into it as a library rather than run as a separate process. Every access to the device passes through that server. A debugger library sits beside it inside the same program and supplies the conventional debug primitives. Section 4.3.4 states how the two divide the work.

Captured trace is stored on the device and read back over the connection that already carries every other operation. No trace port and no trace-capable probe are used.

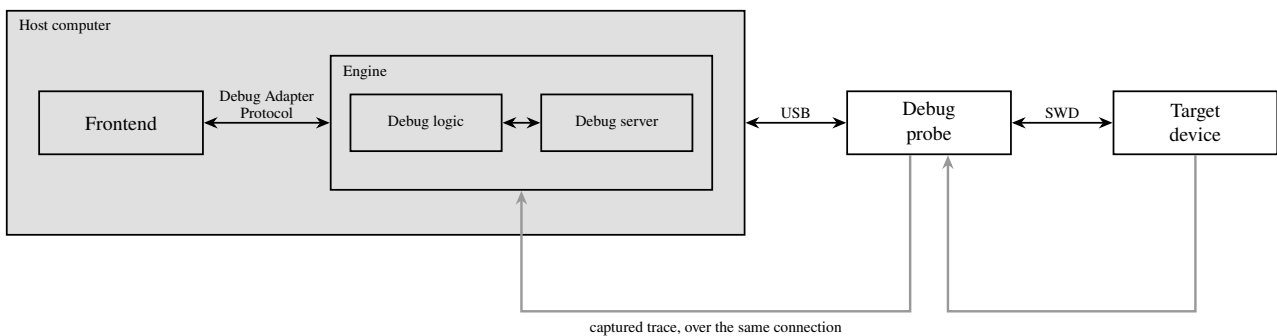


Figure 4.1: Global architecture of the solution

4.1.2 Design Rules

The engine holds every debug capability, and no component of the engine names an interface of a particular development tool. A second frontend can be written against the same engine without reimplementing anything, which is only true if the engine carries no dependency on the first.

Capabilities the standard protocol does not define are exposed through its extension mechanism. Instruction trace, live memory observation, peripheral inspection and module symbols are reached this way.

External constraints shaped the result. Instruction trace and live observation need operations the

conventional debug path does not provide, which produced the division of Section 4.3.4. The libraries used for disassembly and for trace decoding are built with incompatible settings, which restricts how they may be linked, as Section 4.4.2 states.

4.2 Instruction Trace Architecture

This section states the pipeline that turns the execution of a program into a record expressed in source terms. It is the primary contribution of the project.

4.2.1 Pipeline Stages

Trace leaves the processor as a compressed packet stream, and reconstructing execution from it requires the program image.

Generation and capture configure the components on the device and cause trace to be produced and stored. Extraction moves the stored bytes to the host. Decoding converts those bytes into trace elements. Reconstruction and correlation convert those elements into executed instructions, each attributed to a source location.

Each stage depends on one thing. Generation and capture depend on the device and on the debug server. Extraction depends on the debug server. Decoding depends on the trace protocol. Reconstruction depends on the program image and its debug information. The last two stages therefore run on a stored capture with no device present.

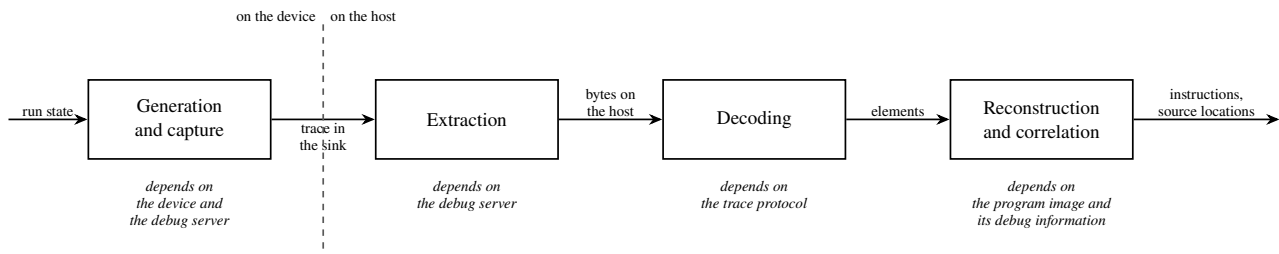


Figure 4.2: The four-stage instruction trace pipeline

4.2.2 Generation and Capture

The trace source is the trace unit paired with the processor. The trace sink is a buffer operating as an on-chip store. An on-chip sink removes the need for trace pins and for a probe able to capture them.

Neither component is commanded by the host while the program runs. Both follow the run state of the processor through event callbacks of the debug server. They are armed when the processor is about to resume, and capture stops when the processor halts. Configuration therefore happens while the processor is stopped, and the host issues no command during a run.

4.2.3 Extraction

Extraction begins when the processor halts and capture has stopped. The fill level of the buffer is interrogated to determine how much to read. A buffer that has wrapped is read whole, since every word then holds data. The buffer is read through a single register that returns successive words, and the result reaches the engine as a callback.

A fixed synchronization pattern is emitted ahead of every payload. A decoder starting at that point recovers alignment with no dependence on what preceded it. Each capture is therefore decodable on its own.

4.2.4 Decoding

Decoding is performed by an external library. The decoder is assembled for each capture from the deformatter configuration, the configuration of the trace unit read from the device itself, the program image, and a sink that receives the decoded elements.

The engine retains instruction ranges, exceptions, exception returns, indications that tracing has started and indications that synchronization was lost. Timestamps and cycle counts are discarded at this boundary.

4.2.5 Reconstruction and Source Correlation

The decoder reports execution as ranges, each naming a start address, an end address and a count. Expanding a range into instructions requires the length of every instruction within it.

The engine disassembles the whole image once, when it is loaded, and records for every instruction its address, length, printed form, control flow classification and source location. Reconstruction is then a sequence of lookups. Source correlation happens in that same pass. Every instruction carries the chain of source locations obtained from the debug information, innermost first, so an address inside an inlined expansion reports both the expanded function and the call site.

Three transformations are applied. The first expands ranges into individual instructions. The second groups consecutive instructions sharing a source location into line blocks, and consecutive line blocks sharing a function into function blocks. The third detects discontinuities and represents them explicitly.

4.2.6 Trace Data Model

The stages exchange value types. A reconstructed instruction records one executed instruction. A function block groups consecutive line blocks belonging to one function, and a line block groups consecutive instructions sharing one source location. A gap records a discontinuity and carries the reason for it.

The model is chronological and holds no deduplication. A loop executed ten times produces ten blocks. The record states what happened.

4.2.7 Trace Lifecycle

Trace is a series of captures, each bounded by a halt.

Trace is unavailable when the device provides no usable components. It is idle when the components exist and tracing has not been requested. It is armed when tracing has been requested and the processor has not resumed. It is capturing while the processor runs. It returns to armed after a halt, once the buffer has been drained.

A reset returns the trace components to their unconfigured state. The pipeline is torn down and rebuilt from the idle state, and the accumulated record is discarded, since it describes a program run that no longer exists.

A buffer that fills during a run loses its oldest data, and the loss appears in the record as a gap. The boundary between two captures is itself a discontinuity, since the processor was halted between them and nothing was recorded during that interval.

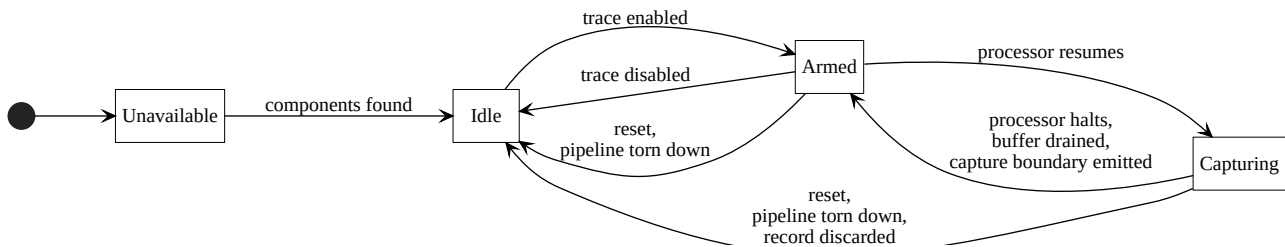


Figure 4.3: The trace lifecycle

4.3 Debugger Architecture

The pipeline requires a host that configures a device, halts it, reads its memory and loads its program image. Providing those capabilities amounts to building a debugger. Making the result usable required the conventional debug primitives alongside them.

4.3.1 Layers

The protocol types layer defines the protocol messages as value types. The protocol transport layer carries them and performs framing, dispatch and cancellation. Neither depends on a debugger library.

The handler layer implements one class per protocol request. A handler parses its arguments, calls into the core layer, and returns a response.

The core layer holds the debug logic. It depends on the protocol types, since the values it produces are the values that travel on the wire. It does not depend on the transport, so a frontend speaking another protocol could reuse it.

The provider layer wraps the external systems the engine drives, presenting each behind a narrow

interface.

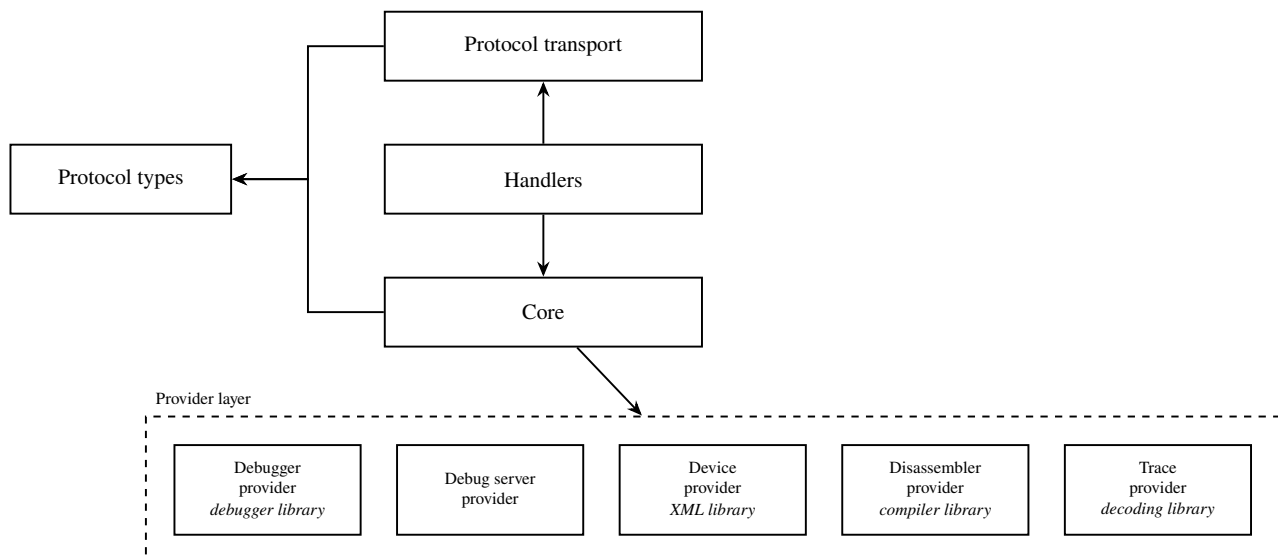


Figure 4.4: Module dependency graph

4.3.2 The Debug Context

A single object composes the core layer. It owns the event bus, the debugger provider, the debug server provider and ten components, and every handler holds a reference to it.

Each component owns one area of debug state: breakpoints, memory, disassembly, modules, variables and expressions, execution control, session lifecycle, live observation, device and peripherals, and instruction trace. Components reach one another through the composing object. The disassembler, device and trace providers are owned by the components that use them.

4.3.3 Providers

A provider adapts an external system to the needs of the engine. The debugger provider holds the debugger and target objects together with the single operation that serializes access to them. The debug server provider wraps the debug server, exposing memory access, command execution, trace component management and target state subscription. The device provider supplies the memory map and the peripheral description. The disassembler provider loads a program image and produces the precomputed instruction table. The trace provider owns the decoding and reconstruction stages. No type belonging to an external library appears in any of these interfaces.

4.3.4 Division of Work Between the Debugger Library and the Debug Server

The engine drives two systems against one device. Neither alone provides what the project requires.

The debugger library supplies the conventional primitives already implemented, covering the interpretation of debug information, expression evaluation, stepping at source granularity and stack unwinding. Reimplementing these would have consumed the time available without producing anything the project set out to contribute.

The library reaches the target over the GDB remote serial protocol, served by the debug server inside the same process.

That protocol does not carry what the trace pipeline needs. Its escape hatch transports a server command as text and returns an unstructured reply, while the decoder must be configured from typed register values read out of the hardware. A capture arrives as a push from the server at the moment of a halt, and the protocol defines no channel for delivering one. The engine therefore also calls the debug server directly, through its provider, and the debugger library is unaware of those calls.

A direct call does not require the processor to be halted. Reading memory through the debugger library requires a stopped program, since the protocol carrying that read reports only when the program is stopped. A direct read is performed by the debug hardware while the processor runs. Live observation of memory and of peripherals depends entirely on that distinction.

Run control belongs to the debugger library. Every request to continue, pause or step passes through it, and the debug server carries those requests.

Memory access is selected when the session is established. A session the engine launched installs the direct path, and every read and write then uses it. A session attached to a server the engine did not create has no direct path, so trace, live memory observation and peripheral inspection are unavailable in that mode.

Reset is performed on the direct path, because the reset behavior required is not reachable through the conventional one. The debugger library is then holding a view of a program that no longer exists, and a mechanism is required to inform it. Chapter 5 describes that mechanism.

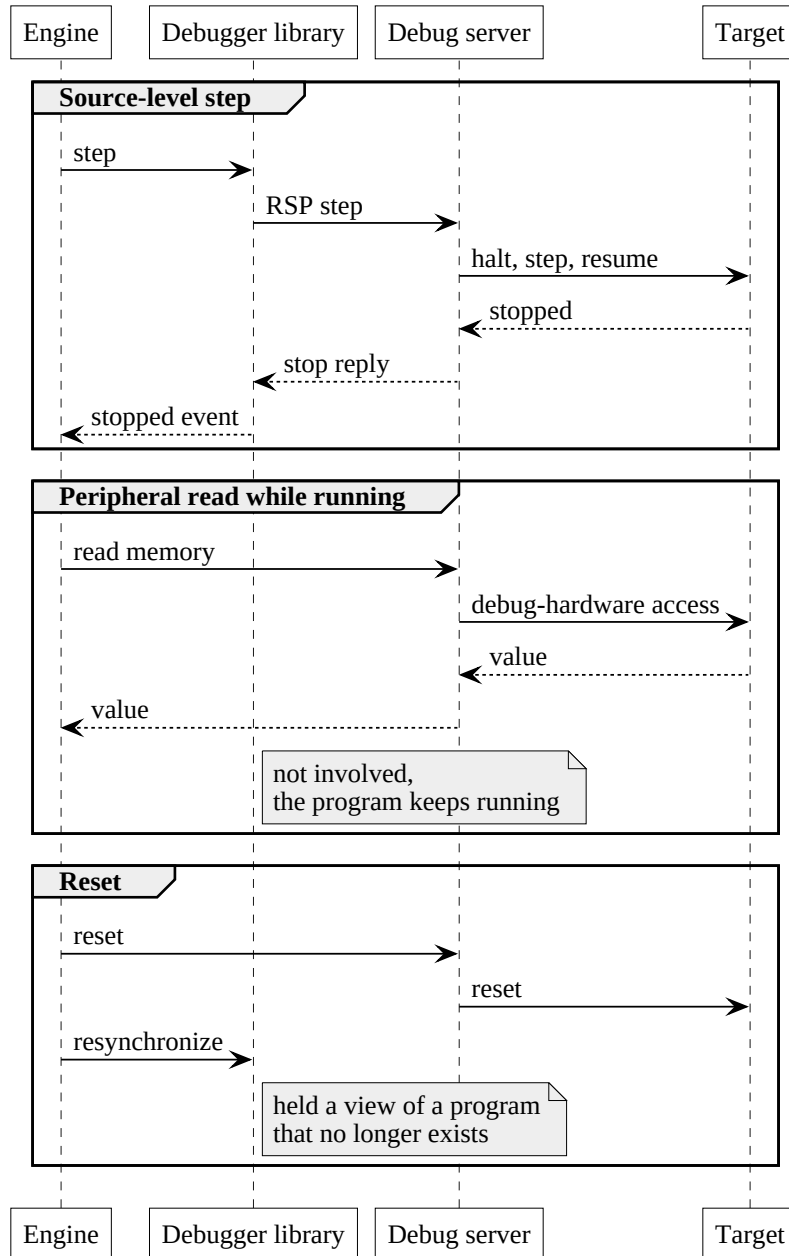


Figure 4.5: Division of work between the debugger library and the debug server

4.3.5 Concurrency

The engine is multi-threaded by necessity. Protocol messages must be read while a request is being handled, so that cancellation is possible. Observation of memory and peripherals must continue while the program runs. The debugger library delivers its events on a thread of its own.

One thread reads messages from the wire and one executes them. A cancellation is recorded as soon as it is read, and execution remains single-threaded, so no two handlers run at once. The debug server runs a loop on its own thread, and every server operation executes there. Worker threads perform observation, trace decoding, program analysis and device description parsing.

The debugger library is not safe against concurrent use. Access to its target object is serialized by

a single operation that takes a callable and runs it under the lock, and the lock is not otherwise reachable. Observation workers read through the debug server and never touch the debugger library, which is what allows observation to proceed while a request is being handled.

Work performed on a background thread must reach the frontend, and the threads that produce it do not construct protocol messages. A producer publishes a typed value describing what happened, and a single subscriber converts that value into a protocol event and sends it. Events are serialized onto the wire in one place, and every producer stays free of any knowledge of the wire format.

4.3.6 Protocol Layer

Messages are read by one thread and executed by another, which is what makes cancellation possible. Capabilities are assembled from the handlers themselves: each handler states what it provides, and the set sent to the frontend is the union. Capabilities the standard protocol does not define are carried by extension requests and events, covering instruction trace, live memory observation, peripheral inspection and module symbols.

4.3.7 Frontend

The frontend is an extension for a development tool. A session object tracks which capabilities the engine advertised and enables the matching views. Each view holds a model that accumulates what the engine sends and a presentation that renders it. The engine sends increments, so the cost of an update follows the size of the change.

4.4 System-Wide Rules

4.4.1 Error Model

A failed operation produces an error response and the process continues. A device may be absent, held in reset, protected against connection or unresponsive, and none of those conditions is a defect of the engine.

4.4.2 Module Boundaries

No component of the core layer constructs a protocol message. Components publish typed values and one place converts them. Without that rule every background thread would need to know the wire format, and a change to that format would reach into every component.

The compiler library used for disassembly is built without run-time type information. The trace decoding library requires it. The two cannot be combined within a single translation unit. The disassembler provider is the only component that touches the first, and its interface exposes none of its types. The trace decoder links the second privately. The pipeline is split across two components for this reason as well as for the separation

stated in Section 4.2.

4.5 Conclusion

The instruction trace pipeline is the contribution of the project. Its stages are separated so that each depends on one thing, which is the device, the debug server, the trace protocol, or the program image.

The debugger is the vehicle. Its structure follows from driving a debugger library and a debug server against one device, so that the capabilities each provides are available without the limitations of either.

Consequences of that arrangement recur in the next chapter. Observation continues while the program runs, which makes live memory and peripheral views possible. Reset is performed outside the conventional path, which makes resynchronization necessary. Chapter 5 describes how each was realized.

IMPLEMENTATION

Contents

1	Development Chronology	49
2	The Application Note	49
3	Instruction Trace Implementation	50
4	The Debugger	55
5	Testing and Validation	64
6	Conclusion	70

Introduction

This chapter states how the architecture of Chapter 4 was realized. It begins with the order in which the work was carried out, treats the three deliverables in turn, and closes with the tests performed and the results obtained.

5.1 Development Chronology

The project did not proceed in the order in which its results are presented. Several architectural decisions of Chapter 4 were reached by discovering that an earlier arrangement did not work.

The work began as a study of the trace infrastructure, which produced the application note of Section 5.2, and as an offline tool that decoded a previously captured trace file against a program image. That tool established the decoding and reconstruction stages, and it required no device.

The second phase addressed acquisition. Obtaining a capture without manual intervention required the trace components on the device to be configured and drained, which the available debug server did not support. The drivers of Section 5.3.1 were written for this purpose.

The third phase turned the result into a usable tool. A command-line program that printed a reconstructed trace demonstrated the mechanism without making it usable during development. The offline tool was absorbed into a debug engine, and the debug server was linked into that engine.

The fourth phase added the remaining debugger capabilities, namely breakpoints, memory observation and peripheral inspection, and corrected the defects found by driving the result against real hardware. The fifth addressed portability and packaging.

5.2 The Application Note

The information required to use instruction trace on these devices is correct, complete and distributed across documents written for different readers. The architecture specifications describe what the components do without describing any particular device. The reference manuals describe a device without describing the trace protocol. A developer must assemble a procedure from several documents, and the assembly is the difficult part. The audience is an embedded developer who has a device, a probe and a debug session, and who wants execution history.

The first part surveys the trace infrastructure of the devices and restates in one place what the specifications and the reference manuals state separately. The distinction that matters most is the one between exporting trace through dedicated pins and capturing it on chip. The exported path requires pins present only on certain packages, requires those pins to be routed on the board, and requires a probe able to receive them. The on-chip path requires none of these and is limited by the size of the buffer. A reader can therefore determine

whether the capability is reachable with the hardware already available.

The second part walks through configuring the trace source and the trace sink, arming them, obtaining a capture, and decoding it against the program image. The difficulty of the task lies in the ordering and in the preconditions. A component that is locked rejects configuration silently. A component configured while the processor runs may be configured inconsistently. A capture decoded against an image other than the one that produced it yields output that is wrong instead of absent. The walkthrough states each condition at the point where it applies.

5.3 Instruction Trace Implementation

5.3.1 CoreSight Trace Drivers

The debug server already carries trace support. It implements a trace macrocell driver limited to the earlier architecture and reached over a JTAG scan chain, a driver for the legacy embedded trace buffer that the Trace Memory Controller replaced, and a driver for the trace port interface unit. None of these reaches an ETMv4 trace unit or a Trace Memory Controller, which are the components this device implements. Two drivers were written and added to the server. Each is an object type created and configured by a configuration script and reachable by name.

5.3.1.1 Sink Driver

The sink driver represents the trace buffer. Its state comprises the access port it is reached through, its base address, the configuration it has been asked to apply, and the state of the capture.

The driver determines what it is attached to. It reads the identification registers and derives from them which configuration the component implements, then checks that result against the component type the same registers report. A mismatch is reported, since proceeding would produce a capture that could not be interpreted.

5.3.1.2 Source Driver

The source driver represents the trace unit, and its initialization proceeds in an order dictated by the hardware.

The component is unlocked first, since a locked component accepts writes and discards them. Its architecture identification register is read and checked, so that a misconfigured base address is reported instead of producing an inexplicable failure later. Its capability registers are then read in a single batched transaction and decoded into the set of features the unit implements, including the number of comparators, the widths of the address and context fields, and which optional features are present.

Reading the capabilities before configuring allows the driver to work on more than one device. The configuration is adjusted to what the unit provides, so a request for a feature the unit lacks is rejected at

configuration time with a message naming the feature.

5.3.1.3 Configuration Validation and Commit

Configuration is staged and then committed. A staged configuration is validated against the discovered capabilities. A validated configuration is written to the device as a batch of queued writes followed by a single completion, which reduces the number of round trips over the debug connection.

The write order is fixed, since several registers are meaningful only once others hold particular values. Where the unit may still be running, at initialization and after a reset, it is stopped first and the driver waits for it to report that it has become idle. On the path taken when the processor resumes, the unit is already idle from the preceding halt.

One capability is never enabled. Tracing of conditional non-branch instructions is supported by the hardware and rejected by the decoding library, so the driver leaves it disabled and records the reason.

5.3.1.4 Command Surface and Run-State Coupling

Both drivers expose commands for creation, configuration and enabling. Creation states the access port and the base address. Configuration sets the options the driver supports, each named. Enabling records an intention and performs no hardware access.

Neither driver is commanded by the host while the program runs. Both register callbacks against the processor target and act on its transitions. When the processor is about to resume, a driver that has been asked to trace validates its configuration, commits it and starts the component. A step is treated differently by the two drivers, since the source is started before the step is taken and the sink afterwards, which keeps the sink disarmed while the processor runs under the control of the debugger. When the processor halts, the source stops the unit and the sink stops capturing and drains its buffer. When the device leaves reset, both drivers reapply their configuration, since reset returns the components to their locked and unconfigured state.

Trace therefore follows the program with no host action during a run, configuration always happens while the processor is stopped, and reset is handled. An unconfigured trace unit after a reset was the cause of one of the defects recorded in Section 5.5.4.

5.3.2 Trace Extraction

Extraction runs when the processor halts. Its difficulty is determining how much of the buffer holds valid data.

Capture is stopped before it is read. The driver requests that the component stop on flush, issues a manual flush, then waits for the component to report readiness. A component reporting an empty buffer is handled separately, since draining an empty buffer would produce a zero-length capture that the decoder would treat as a stream.

The quantity to read is obtained from the component. The current fill level is read, and where the component reports that the buffer has wrapped, the whole buffer is read instead, since every word then holds data and the fill level no longer describes the extent. The fill level is meaningful only while capture remains enabled, so the level is read before capture is disabled, and capture is disabled only after the drain has completed.

The buffer is read through a single register that returns successive words and advances an internal pointer. The read is issued as a repeated access to one address with the automatic address increment of the access port suppressed.

The extracted block is delivered to a callback registered by the engine and, where capture to the host is enabled, to a file or a network connection. The engine uses the callback. The second destination allows a capture to be obtained without the engine, for the purposes of Section 5.5.

A fixed pattern of synchronization bytes is emitted ahead of every payload, on every destination. The pattern occupies one whole formatter frame and causes a decoder to reset its state. Each capture is therefore decodable on its own, and the successive captures of a session carry no state between them.

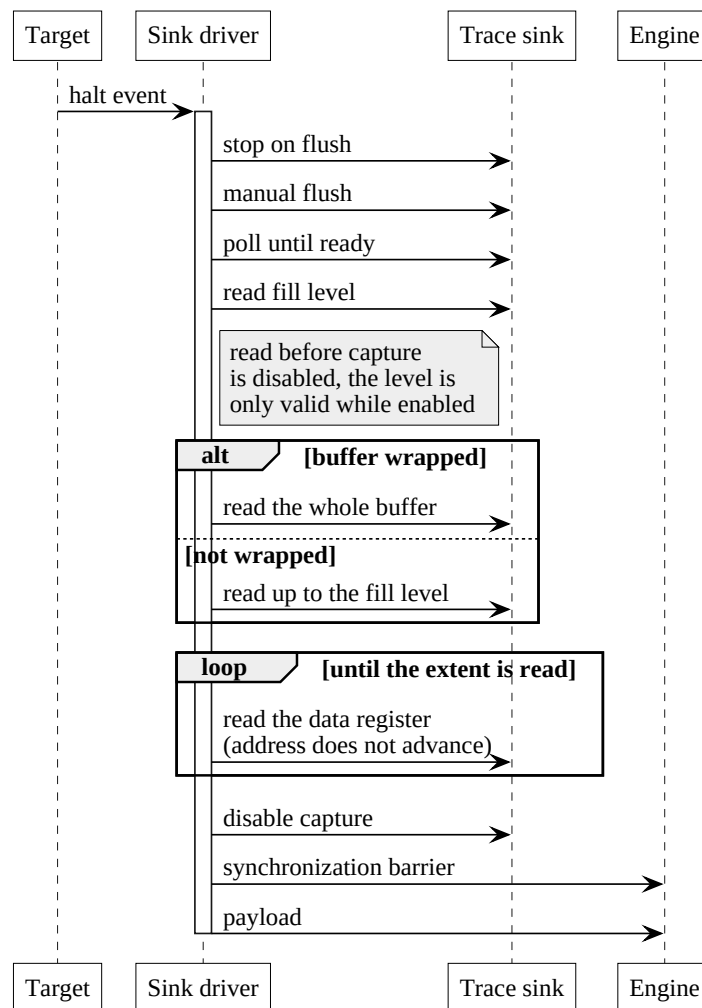


Figure 5.1: Extraction sequence

5.3.3 Trace Decoding

Decoding is performed by an existing library, and the implementation consists of configuring it correctly for each capture.

The inputs to this stage and to the reconstruction that follows are a captured byte stream, the program image, and the register values recorded from the trace unit. None requires a device or a live session, so a capture may be decoded after the session that produced it has ended.

A decode tree is constructed for each block. It is told that the stream is frame-formatted, that frames are aligned to memory, and that four consecutive synchronization patterns reset its state, which is the property the barrier of the preceding subsection requires.

The decoder must be configured exactly as the trace unit was. The identification registers are read from the unit itself, so the decoder is configured against the capabilities the hardware reports. The control register and the trace identifier register hold the values the driver committed, which the commit path of Section 5.3.1 has already validated against those same capabilities. A decoder configured from an assumed capability set would misinterpret every capture on any device that differed from the assumption, and would do so without reporting an error.

The program image is registered with the decoder as the source of instruction bytes, described by its loadable segments. Each segment states an address on the device, an offset within the file and a length. The decoder needs those three values in order to answer a request for the bytes at a given address.

The decoder reports elements to a sink object. The sink retains instruction ranges, which carry the executed addresses, exception and exception return elements, and trace-on and loss-of-synchronization elements, which mark discontinuities. Everything else is discarded. Each retained record carries the byte offset within the capture at which the originating packet began, and the trace identifier of the stream it came from. These allow a reconstructed instruction to be traced back to the bytes that produced it.

5.3.4 Static Program Analysis

The image is analyzed once when it is loaded, and the result serves both reconstruction and the breakpoint pruning of Section 5.4.5.

The image is opened as an object file and its loadable segments are extracted, skipping those with no content in the file. The segments are registered with the decoder and identify which parts of the file correspond to addresses on the device.

Mapping symbols are collected and converted into a list of spans, each stating an address range and the instruction set in force within it. Spans marked as data are skipped entirely, which prevents literal pools from being decoded as instructions. A section carrying no mapping symbols is treated as a single span of the default

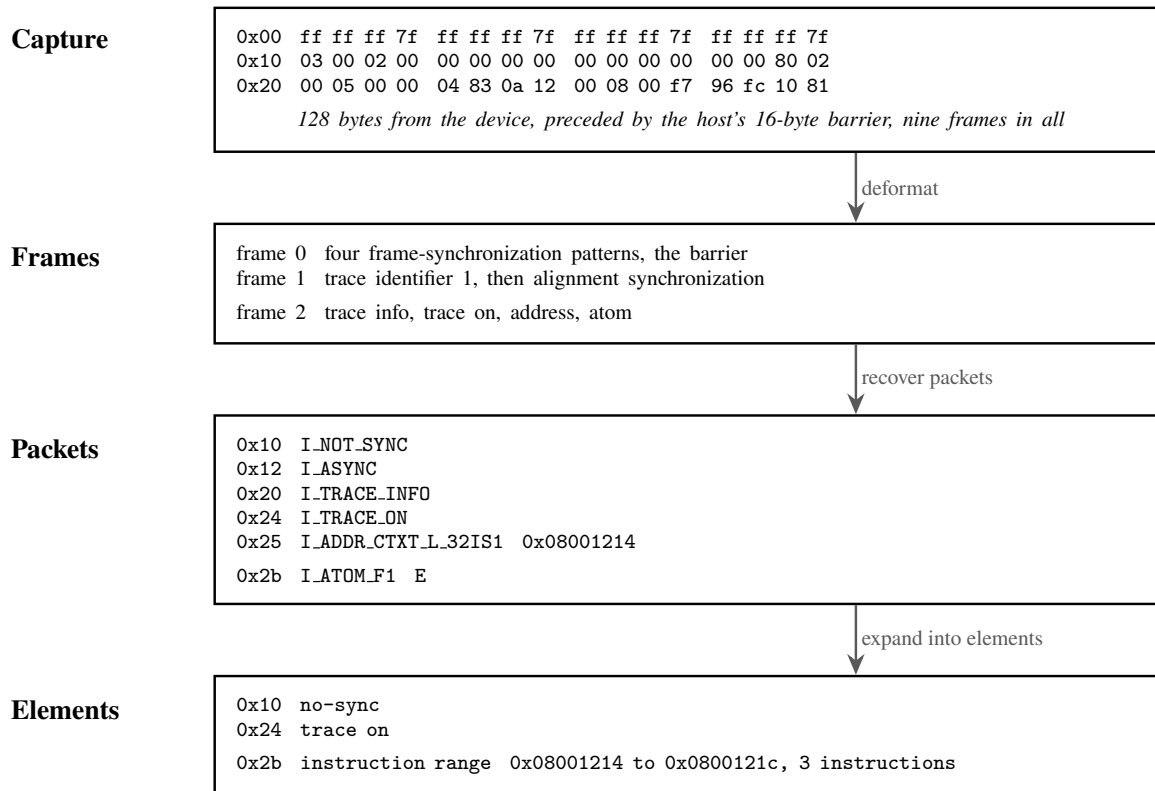


Figure 5.2: From bytes to elements, on the capture held as test data

instruction set, and bytes preceding the first mapping symbol are left uncovered.

Each span is decoded instruction by instruction. For every instruction the analysis records its address, its length, its raw bytes, its mnemonic and operands, and a classification stating whether it is a branch, a call, a return, conditional or indirect. Where a branch target is determinable from the instruction itself, it is recorded. The classification is obtained from the analysis facility of the compiler library, with one addition. That facility does not recognize as returns the instruction forms a compiler for this architecture emits for a return, so those forms are matched on the printed instruction. The addition is a known weakness and is recorded in Section 5.5.6.

The source location of every decoded instruction is resolved during the same pass. The whole chain of inlined frames is requested, and each frame records a function, a file, a line and a column. Resolving during the analysis pass keeps reconstruction inexpensive, since the cost is paid once per image and never per instruction executed.

5.3.5 Reconstruction Transforms

The decoded elements are converted into the model of Section 4.2 by transformations expressed over the same record sequence.

A range is expanded by walking from its start address to its end address, taking the length of each instruction from the precomputed table. An address the table does not contain terminates the expansion of that

range and is reported, since continuing would require guessing an instruction length. Where the decoder reports that the last instruction of a range was not executed, which occurs for a conditional branch that was not taken, the transformation marks it and keeps it. The instruction is dropped one layer higher, where the increment is assembled, so the output of the transformation remains a complete account of what the decoder reported. The instruction set reported by the trace is compared against the instruction set recorded by the static analysis, and a disagreement is reported, since it indicates that the image and the capture do not correspond.

Consecutive executed instructions sharing an identical chain of source locations are merged into a line block, and consecutive line blocks belonging to the same outermost function are merged into a function block. Comparison uses the whole chain, so two instructions inlined from different call sites are not merged. Grouping is chronological and performs no deduplication. Sources written in assembly are excluded from merging, since one line of assembly corresponds to one instruction.

Discontinuities are detected by maintaining a running count of executed instructions and emitting a gap wherever the element stream indicates one. The causes distinguished are buffer overflow, the resumption of tracing after it had stopped, and loss of synchronization. A fourth cause is added outside the transformation, since every capture after the first is preceded by a boundary gap. A boundary produces more than one marker, because the synchronization barrier that precedes every payload resets the decoder, so the stream opens with a loss of synchronization followed by a resumption of tracing. The view presented to the developer collapses markers falling at the same instruction index into one, and the record retains each cause.

5.3.6 Delivery to the Frontend

The result is sent to the frontend as an increment. Each message states the index at which its contents begin, followed by the instructions, blocks and gaps the capture produced.

The frontend validates that the stated index matches the length of what it already holds, and discards an increment that does not match. An increment applied at the wrong index would produce a record that is plausible and wrong.

5.4 The Debugger

5.4.1 OpenOCD as an In-Process Library

Section 4.3.4 established that the engine must call the debug server directly. The server is a program, so making it a library required four changes.

5.4.1.1 Build System Migration

The build system of the server was replaced so that it could be built as a static library and consumed by the build of the engine. Each group of sources became an object library, and the groups were aggregated into one library with no entry point. The source file containing the entry point remains in the tree and is not built, since the engine supplies the entry point.

5.4.1.2 Containing Process Termination

A program may terminate the process when it encounters a condition it cannot handle, and the server does so in many places. A library may not, since a condition affecting one device must not end a debug session.

Every termination call in the server was redirected to a replacement function, imposed on every translation unit by the build. The replacement examines a per-thread state. Where a guarded call is in progress, it performs a non-local jump back to the guard, which returns the termination code to the caller as a failure. Where no guarded call is in progress, it terminates as before.

Every entry into the server from the engine is wrapped in such a guard, so a condition that would have ended the process becomes a value describing which operation failed and with what code. Destructors are not run on the abandoned call chain, so the guard is applied around calls into the server and not around code of the engine. Allocations the server made on the abandoned call chain are leaked, and a change it had partly completed is not undone. The mechanism is a containment of last resort.

5.4.1.3 Embedding the Startup Scripts

The server executes a set of scripts at startup that define much of its command surface, ordinarily read from disk. They are converted into arrays at build time and compiled into the library, so the engine has no dependency on their location.

5.4.1.4 Runtime Initialization

The initialization the entry point of the server would have performed is reimplemented by the engine. A command context is created, the logging subsystem is started, and the command sets of each subsystem are registered in the order the original uses, since later registrations depend on earlier ones.

The startup sequence then follows the order the server requires, namely initializing targets, the adapter and the transport, initializing the debug access port, examining the targets, initializing the flash and trace subsystems, and starting the protocol server that the debugger library connects to.

5.4.1.5 Thread Affinity and Command Serialization

Every operation of the server executes on the thread running its loop, and that loop waits on sockets. Work submitted from another thread must reach that thread and must wake the loop, which will otherwise wait indefinitely.

An additional service is registered with the server, bound to a port on the loopback interface, and a client is connected to it. The loop already waits on the sockets of its services, so sending one byte on that connection wakes it, and the handler for the service drains a queue of pending work and executes it. Every public operation of the provider submits work to that queue and waits for the result, with a bounded timeout so that no caller can block indefinitely.

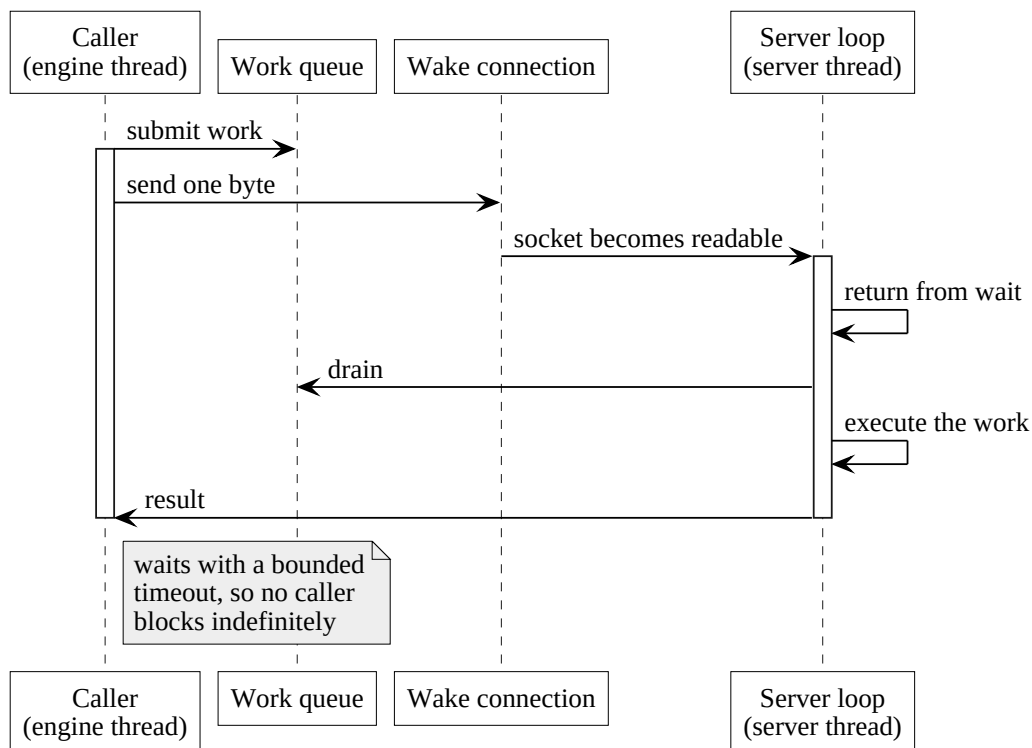


Figure 5.3: Command queue and wake mechanism

5.4.2 LLDB Integration

The debugger library exposes its capability through a stable interface intended for embedding, in which each class is a handle to an internal object. The engine drives that interface directly, using the classes covering the debugger, the target, the process, threads, stack frames, values, breakpoints, watchpoints, modules, addresses, symbols and line entries. The interface gives structured results, and a value obtained through it carries its type and its children, which the variable presentation of Section 5.4.9 requires.

The library connects to the target over the GDB remote serial protocol, served by the debug server running inside the same process. The connection is made over the loopback interface, so the two components

exchange protocol messages exactly as they would if the server were a separate process. The engine never constructs a message of that protocol itself.

Serialization of target access is implemented as a single template operation that acquires the lock of the library and invokes the supplied callable. The lock is not reachable from outside. Components holding only a reference to the composing object reach the operation through it.

Settings of the library are changed at startup, each for a reason established by testing. External symbol lookup is disabled, since the library otherwise attempts to fetch symbols from network services when it encounters an image with partial debug information, which delays the creation of a target considerably. Parallel loading of modules is disabled, since the background workers it creates acquire locks in an order opposite to that used by the serialization above, producing a lock order inversion that was confirmed with a thread sanitizer against real hardware. The strategy for resolving breakpoints at inlined call sites is fixed, so that a personal configuration file on a developer machine cannot silently change where breakpoints are placed.

One change to the server was required by this pairing. The server routed every packet beginning with a particular letter to a handler for an obsolete feature, while the library sends several unrelated packets beginning with that letter. The server now matches the complete packet name and replies to the others with the empty response that indicates an unsupported packet. A second change concerns reset and is described below.

5.4.3 Session Lifecycle

Launching creates the debug server, installs the memory access strategy that uses it, creates the target from the program image, connects the debugger library to the server, and starts the trace pipeline. Failure of the trace pipeline does not fail the launch and is reported to the developer as a message, since a device without trace components remains usable as a debug target.

Attaching connects to a target already running under a server the engine did not create. Every capability depending on the direct path is unavailable in this mode, including trace, live memory observation and peripheral inspection.

Disconnection proceeds in a fixed order. The program is detached or terminated, event handling is stopped, the trace pipeline is shut down, the observation workers are stopped, and the debug server is shut down last, so that no thread outlives the resource it reads from.

5.4.4 Reset and State Resynchronization

Resetting the device is the operation where the debugger library and the debug server conflict, and resolving that conflict required a change to the server.

The reset is performed on the direct path, because the behavior required, which is to reset and halt the device, is not reachable through the conventional one. The processor therefore halts without the debugger

library having asked for anything, and the library continues to believe that the program is where it last saw it. Its cached view of threads, frames and variables is stale, and a developer inspecting them after a reset would be shown values from a program run that no longer exists.

The library learns that a program has stopped only from a reply to a request it made. The server reports a stop only when it believes the program was running. Neither condition holds after an out-of-band reset.

The change adds a command to the server that arms a flag on the connection, together with an event that carries it. When the flag is armed, the next resume or step request received on that connection is answered with a report of the current state instead of being executed. The engine arms the flag after the reset and then issues a single instruction step, which the server intercepts and answers without stepping the device. The library receives the reply, updates its view, and the session continues against the program that is actually present.

The sequence also handles trace. The pipeline is shut down before the reset and rebuilt after it, since the reset returns the trace components to their unconfigured state, and the accumulated record is discarded.

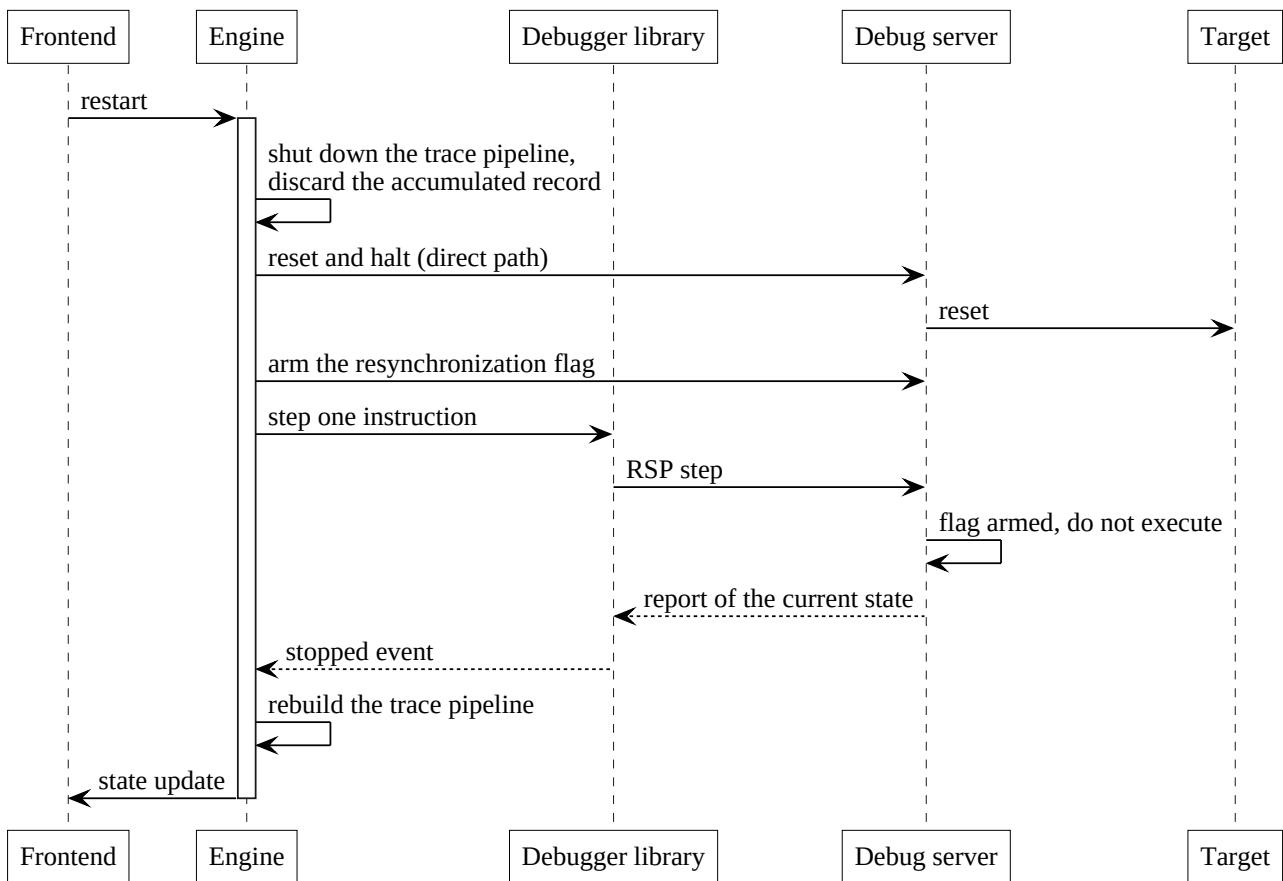


Figure 5.4: Reset and resynchronization

5.4.5 Breakpoints

The choice between a software and a hardware breakpoint follows from the memory region the address lies in, and the engine makes that choice automatically.

For each breakpoint the engine consults the memory map supplied by the device provider. Where every

location of the breakpoint lies in writable memory a software breakpoint is used, which conserves the limited comparators. Otherwise a hardware breakpoint is used, since a software breakpoint placed in flash memory is a write that has no effect and produces a breakpoint that never triggers. Where no memory map is available, a hardware breakpoint is used, since that choice fails visibly.

The developer may override the decision, and the setting distinguishes an explicit choice from the absence of one, so that a default does not silently defeat the automatic behavior.

Refinements were added after testing. A hardware breakpoint that cannot be allocated is reported against that breakpoint instead of failing the request, and the message produced by the library is replaced by one stating that the comparators are exhausted. Breakpoint locations that do not correspond to a decoded instruction are disabled, using the static analysis above, which prevents a breakpoint from being placed inside a literal pool.

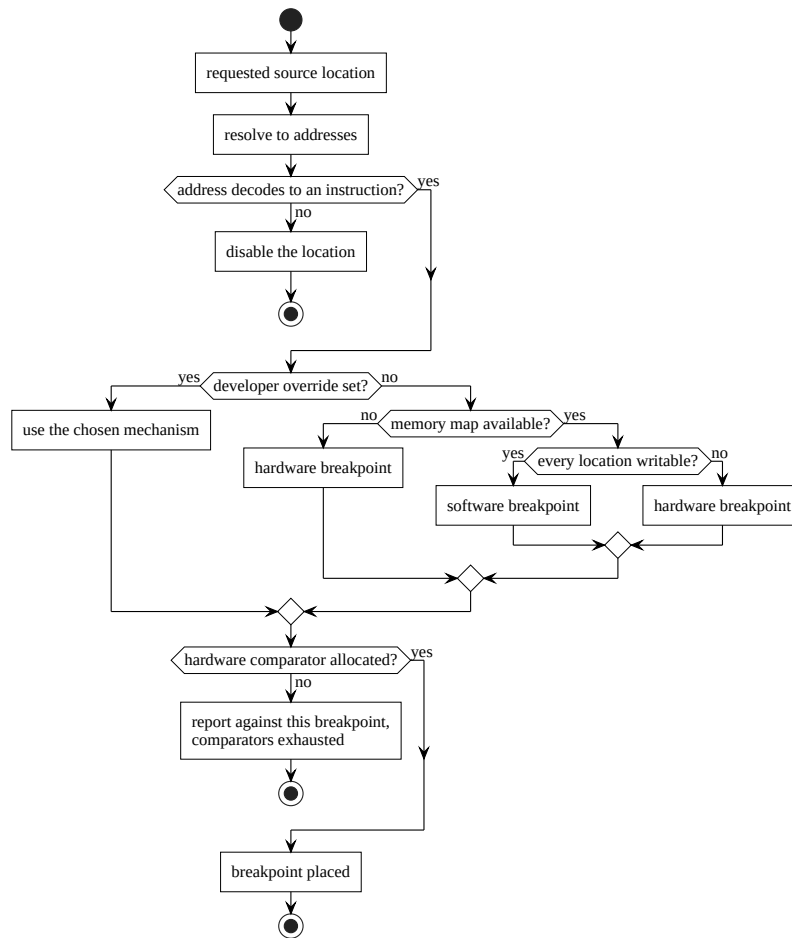


Figure 5.5: Breakpoint placement decision

5.4.6 Memory Access and Live Observation

Memory is read through one of two strategies, selected when the session is established. The strategy using the debugger library serves a conventional session. The strategy using the debug server is installed whenever the direct path is available, since it does not require the program to be stopped.

Live observation is built on the second strategy. Each observation runs on a thread of its own, reads the

requested region at a configured interval, and publishes the result on the event bus. A thread per observation is used because the number of simultaneous observations is small, and the wait between reads is interruptible so that stopping an observation takes effect immediately.

Observations are not suspended while the processor is halted. Peripherals continue to run while the processor is stopped unless they have been explicitly frozen, so a value that changes during a halt is a real observation.

5.4.7 Peripheral Inspection

The device description is parsed from the vendor file into a plain data model of peripherals, registers and fields, each carrying its address, its access permission, its reset value and the meaning of its values. The file may be large, so parsing proceeds on a background thread while the session continues, and the parts required before execution begins are loaded synchronously.

A register whose reading changes the state of the peripheral is not read unless the developer asks for it explicitly, so that opening a view does not perturb the program. Registers that are adjacent and safe to read are coalesced into a single access, which reduces a view of a peripheral to a small number of reads.

A race between a write and a periodic read is resolved on the producing side. A counter is advanced whenever a write completes, and an observation whose read overlapped a completing write is discarded instead of published, so a stale value cannot be presented as current.

5.4.8 Disassembly, Variables and Expressions

Disassembly is implemented twice in the engine, and the reason is the module boundary of Section 4.4.2. The implementation serving the trace pipeline is the static analysis above, which processes the whole image once and exposes no type of the compiler library across its interface. The implementation serving the disassembly view of the debugger uses the facility of the debugger library. Unifying the two was identified during the work and not carried out.

Variables are obtained from the debugger library. The engine converts each value into the representation the protocol defines, recursing into structured values on demand, so a large structure is expanded only where the developer opens it.

References to values are allocated from two spaces, one for values invalidated when the program resumes and one for values that persist. Separating them allows the first space to be cleared on every resumption without disturbing the second.

Expression evaluation is exposed through the same component. A prefix character distinguishes an expression in the source language from a command addressed to the underlying debugger, which gives access to capability the protocol does not describe without adding a request for each.

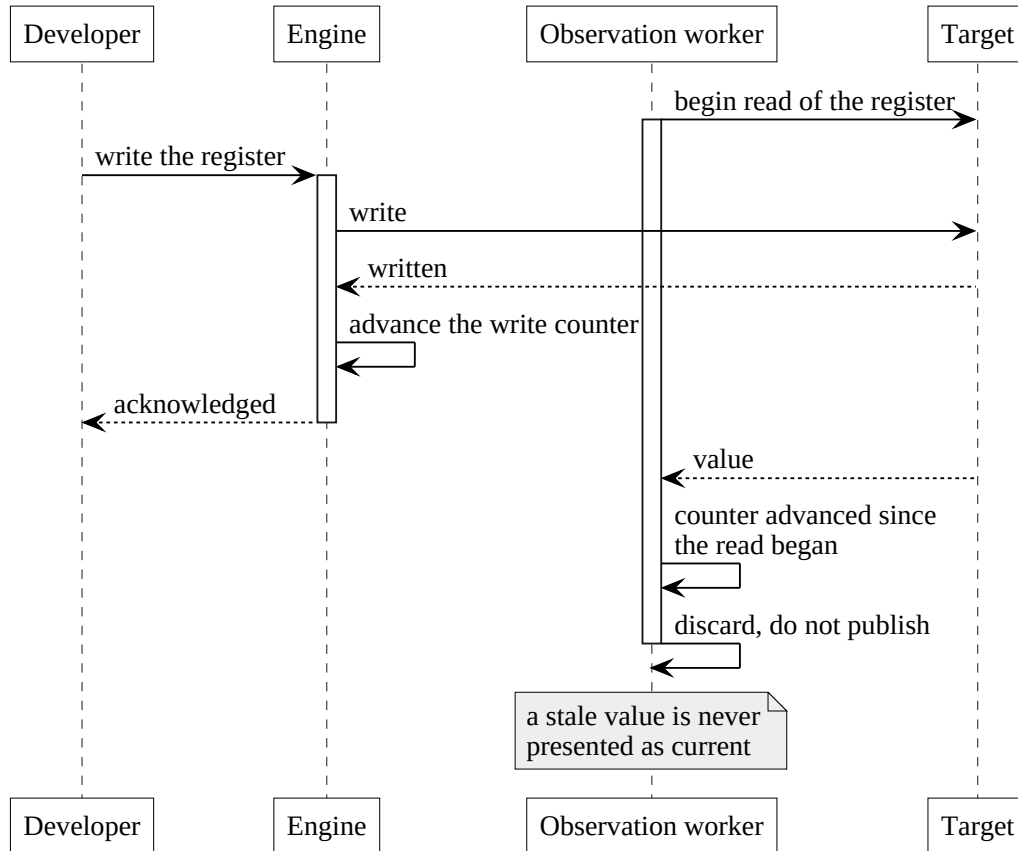


Figure 5.6: Peripheral write and observation

5.4.9 Protocol Server and Custom Extensions

The protocol layer implements the message framing, the dispatch and the handler hierarchy of Section 4.3. The engine registers a handler for each of the requests listed in Table 5.1, together with a catch-all handler for unrecognized requests.

Handlers share a base that parses arguments, invokes the handler body and converts a returned error into an error response, so no handler contains message construction. Two handler shapes exist, one answering immediately and one deferring its response, the second being required by the requests that must not be answered until configuration is complete.

The custom requests are accompanied by events that carry what the engine produces asynchronously, namely trace increments and status, the bytes and state of a memory observation, the values and state of a peripheral observation, and the report that the device was reset.

5.4.10 The Frontend Extension

The frontend contributes the views the standard protocol does not cover. Two views present trace, one grouping the record by function and line and one listing executed instructions in order. A memory view and a set of live memory panels present regions of memory. A peripheral view presents the device description.

Table 5.1: Registered protocol requests by area

Area	Requests	Kind
Lifecycle	initialize, launch, attach, configurationDone, restart, disconnect, cancel	Standard
Execution	continue, next, stepIn, stepOut, stepInTargets, pause	Standard
Breakpoints	setBreakpoints, setFunctionBreakpoints, setInstructionBreakpoints, setDataBreakpoints, setExceptionBreakpoints, breakpointLocations, dataBreakpointInfo	Standard
Inspection	threads, stackTrace, scopes, variables, setVariable, evaluate, completions, exceptionInfo	Standard
Memory	readMemory, writeMemory	Standard
Disassembly	disassemble	Standard
Symbols	modules, moduleSymbols, compileUnits, locations, source	Standard and custom
Trace	trailerTraceEnable, trailerTraceDisable, trailerTraceStatus	Custom
Observation	trailerWatchStart, trailerWatchStop	Custom
Peripherals	trailerDeviceInfo, trailerPeripheralDetail, trailerPeripheralRead, trailerPeripheralWrite, trailerPeripheralWatchStart, trailerPeripheralWatchStop	Custom

The trace views consume the increments described above and render source locations as links, so selecting an entry opens the corresponding source position. Discontinuities are rendered as explicit separators.

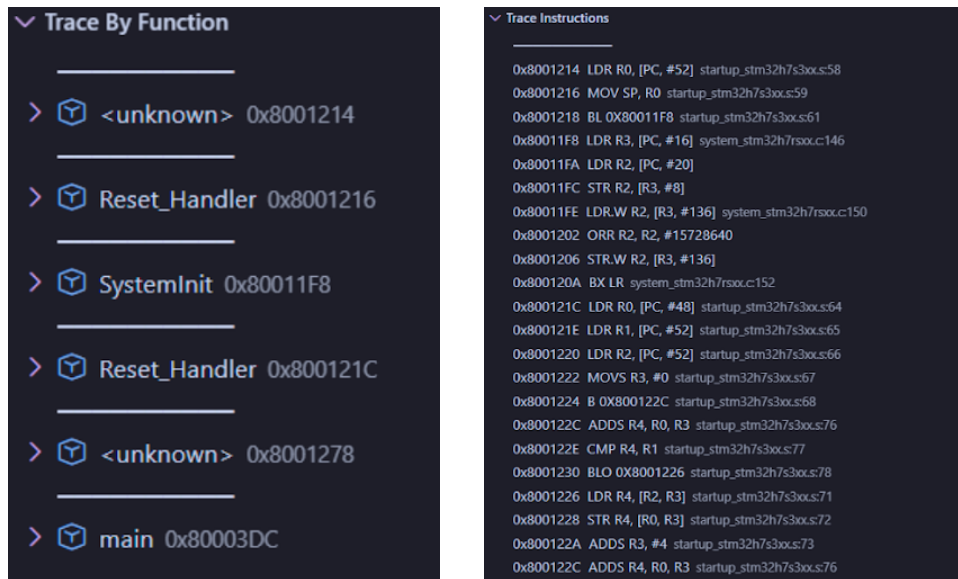


Figure 5.7: The trace views. The record is grouped by function on the left and listed instruction by instruction on the right, each instruction carrying the source location resolved for it.

5.4.11 Portability and Provenance

The engine builds on Linux and on Windows. Portability work concentrated on the reading of protocol messages, which requires a different mechanism on each platform, the resolution of resource paths relative to the executable, and the initialization requirements of the libraries used. An installer is produced for Windows, carrying the

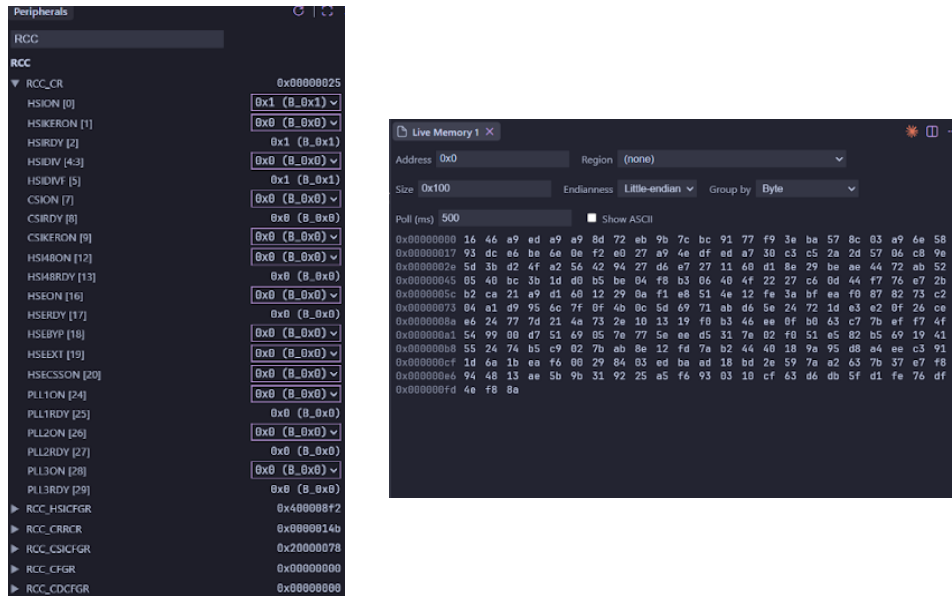


Figure 5.8: The peripheral view with a register expanded to its fields, beside a live memory panel

engine, the resources it requires and the scripts of the debug server. The frontend is not yet distributed through the extension marketplace.

The engine is built on existing open-source work. The protocol layer began as the debug adapter distributed with the debugger library and was restructured into the layers of Section 4.3. The debug server is a fork of an existing open-source server, to which the two drivers of Section 5.3.1 and the protocol corrections above were added. Decoding is performed by an existing library.

Written for this project are the two CoreSight drivers, the trace pipeline from extraction through reconstruction, the domain components that constitute the core of the engine, the device description provider, and the frontend extension. The licences of the adopted work are carried unchanged and differ between components, which constrains how the result may be redistributed.

5.5 Testing and Validation

5.5.1 Test Strategy

Testing was organized in three tiers, distinguished by what each requires in order to run.

The first tier requires nothing beyond the built engine. It covers the protocol handshake, the cancellation path, and the components that operate on stored data, namely the device description parser, the static program analysis, the trace decoder and the reconstruction transformations. This tier runs during ordinary development.

The second tier requires a device. It drives the engine at the level of the protocol with a script that sends requests and checks responses, covering launching, attaching, stepping, breakpoints, peripheral access and trace.

The third tier requires a device and the frontend. It drives the extension inside the development tool

and checks the behavior a developer would observe, including that the views populate and that state is refreshed after a reset. It is exercised by hand.

No unit testing framework is used. Tests in the first tier are plain executables that return a status, which keeps the build free of an additional dependency.

Concurrency is covered separately. A test publishes on the event bus from several threads at once, so that a defect in its locking appears as a detected race instead of an intermittent failure in a larger test. The engine as a whole was additionally built with a thread sanitizer and driven against the device, which is how the lock order inversion of Section 5.4.2 was identified.

Table 5.2: Test inventory

Test	Tier	What it verifies
<code>device_svd_test</code>	One	The device description parser against a vendor file
<code>device_provider_test</code>	One	The memory map and the peripheral model built from that description
<code>program_disassembler_test</code>	One	Spans, instruction decoding and source resolution
<code>trace_decoder_test</code>	One	A full decode of a stored capture without a fatal response
<code>transform_test</code>	One	Instruction expansion and function block grouping over a record sequence
<code>trace_provider_test</code>	One	The whole offline pipeline, against counts frozen from a verified run and against properties checked independently
<code>event_bus_test</code>	One	Publication on the event bus from several threads at once
<code>orchestrator_cancel_test</code>	One	The cancellation path through the dispatcher
<code>llvm_lldb_coexistence_test</code>	One	That the compiler library and the trace decoder link into one program
<code>handshake_test</code>	One	The protocol handshake and the capability set returned to the client
<code>openocd_provider_hw_test</code>	Two	Trace components located, register readback and capture delivery
<code>launch_smoke_test</code>	Two	Threads, call stack, memory reading, disassembly and variable inspection in a launched session
<code>attach_smoke_test</code>	Two	Threads and call stack in a session attached to a running server
<code>stepping_smoke_test</code>	Two	Stepping over, into and out of a call, and the behavior when the comparators are exhausted
<code>peripheral_smoke_test</code>	Two	Peripheral enumeration, register reading and writing, and an observation discarding a frame that overlaps a write

5.5.2 Hardware Test Bench

The bench consists of a development board carrying the target device together with an on-board probe, connected to the host computer by a single cable. It contains no trace probe and no connection to trace pins. Every trace result reported below was obtained over the same connection used for ordinary debugging, which is the objective stated in Chapter 3.

A small firmware project serves as the program under examination, built with debug information and

with the optimization settings a developer would ordinarily use.

Every result reported in this chapter was obtained in a session that the engine launched, since trace, live memory observation and peripheral inspection all depend on the direct path.

The pipeline was additionally exercised on an STM32H7 device and on an STM32N6 device, the second of which is built on a later processor and implements a different trace unit. The base addresses, the configuration script of the family and the intermediate trace links between the source and the sink changed. Neither driver was modified, since each reads the capabilities of the component it drives from that component's own identification registers. The drivers are therefore independent of the device and of the particular implementation of the components. On the device carrying more than one processor, the primary core alone was traced.

5.5.3 Trace Correctness Validation

Correctness of the pipeline was checked against a stored capture, against the device, and against debuggers that produce a reconstruction of their own.

The decoding and reconstruction stages are checked against a stored capture. The capture, the program image and the trace unit register values recorded from the device are held as test data. The test decodes the capture and asserts that the number of executed instructions equals a fixed expected value, that every reconstructed address corresponds to an instruction in the static analysis, and that at least one function block carries a resolved function name. Holding the register values as data allows the test to reproduce the configuration the hardware actually had.

The acquisition stages are checked against the device. A test asserts that the trace components are found, that the register values read from the trace unit match those recorded, and that a capture delivers exactly two callbacks per halt, which are the synchronization barrier followed by the payload.

The reconstruction was compared against the output of commercial debuggers. The reset handler supplied with the vendor firmware package for this family, and a small example program, were traced on the development board. The same execution was captured by three commercial debuggers, each driving the probe of its own vendor. Their decoded listings were exported and compared instruction by instruction against the reconstruction produced here, and the sequences agreed. The differences observed were confined to presentation, since one of those tools marks exception entry and exit and annotates whether a branch was taken. The exported listings were not retained, so this comparison is reported as work carried out during development.

A capture obtained by a probe of another vendor was decoded by the same pipeline. That probe captured the trace of the same program over the trace port, and its raw output was supplied to the decoding stages in place of a capture drained by the driver. Once the synchronization framing is discounted, the byte stream was identical to the one drained from the sink. The stream a trace unit produces therefore does not depend on the sink that absorbs it, and the decoding stages depend on neither the probe nor the path by which the capture was

obtained.

Behavior that no test on stored data reaches was exercised on the device. A long-running program filled the sink, which reported itself full and was then read whole, and the trace unit reported an overflow that appeared in the reconstruction as a gap carrying that cause. A session of consecutive instruction steps accumulated many captures, across which the record stayed coherent and the index of each reconstructed instruction continued to advance. The quantity of execution a full buffer holds was not recorded, so the figure derived from Table 5.5 remains an extrapolation from a single capture.

One weakness should be stated. The stretch of execution from reset to the entry of the main function is fixed startup code and is therefore known in advance, which makes it an exact oracle for a reconstructed instruction sequence. The tests assert an instruction count and a set of general properties instead of that literal sequence. Asserting the sequence would be a considerably stronger guarantee and was not implemented.

5.5.4 Defects Found on Hardware

Driving the result against a real device produced defects that no test on stored data would have found. The cause of each is more informative than the symptom.

Table 5.3: Defects found on hardware

Symptom	Cause	Fix	Section	
A breakpoint on a source line resolved into a vendor header	Several source locations overlap at an inlined call site and the innermost was selected	Walk outward to the outermost inlined block and use the call site recorded there	5.4.5	
Breakpoints could not be set in assembly files	The frontend had not declared that it supports breakpoints in that language	Declare the language	5.4.5	
A restart left a stale view of the program	The reset is performed outside the conventional path, so the debugger library was never told	The resynchronization mechanism	5.4.4	
Variables and scopes were stale after a restart	The same cause	The same mechanism	5.4.4	
Trace was inconsistent and often empty	The components were not armed on every transition that resumes the processor, and reset was mishandled	Couple both drivers to the run state of the target	5.3.1	
Stepping did not halt where expected	The internal breakpoints the debugger library places for a step were software breakpoints in flash, which never trigger	Force those breakpoints to use comparators	5.4.5	

5.5.5 Results

The objective stated in Chapter 3 was met. Instruction trace is captured from an on-chip sink, extracted over the ordinary debug connection, decoded, reconstructed into a sequence of executed instructions and attributed to source locations, and presented inside a development tool during a live debug session. No trace pins and no specialized probe are used.

The debugger that carries this capability provides breakpoints with automatic selection between the two mechanisms, stepping at source and instruction granularity, variable and expression inspection, memory reading and writing, disassembly, device reset, and the extensions covering trace, live memory observation and peripheral inspection. The engine builds and runs on two host platforms, and was demonstrated on both.

Table 5.4: Verification of the requirements

ID	How it was established	Status
FR1	Trace source and sink located and their registers read back against the device	Met
FR2	Capture delivery observed on every halt, two callbacks per halt	Met
FR3	Every result in this chapter obtained over the debug connection alone	Met
FR4	Decode of the stored capture, Figure 5.2	Met
FR5	Every reconstructed address resolves to an instruction in the image, the counts match the frozen baseline of Table 5.5, and the reconstructed sequence was compared against three vendor debuggers, Section 5.5.3	Met
FR6	Function blocks carrying resolved names in the same test	Met
FR7	Gaps counted and represented explicitly in the same test	Met
FR8	Trace views populated during a live session, observed in the development tool	Met
FR9	<code>stepping_smoke_test</code> against the device, covering stepping over, into and out of a call, and breakpoints	Met
FR10	Memory reading exercised by <code>launch_smoke_test</code> against the device, and register access exercised on both paths	Met
FR11	<code>peripheral_smoke_test</code> against the device, covering enumeration, reading and writing	Met
FR12	Disassembly, scopes and variables exercised by <code>launch_smoke_test</code> against the device	Met
FR13	Reset followed by the resynchronization of Section 5.4.4, observed against the device	Met
NFR1	The bench carries no trace probe and no connection to trace pins, Section 5.5.2	Met
NFR2	The engine builds and runs on both host platforms	Met
NFR3	The frontend is an extension speaking the standard protocol	Met

Table 5.5: Measured figures for one capture

Quantity	Value
Interval traced	Reset handler to the entry of main
Trace produced by the device	128 bytes, eight formatter frames
Host-inserted synchronization barrier	16 bytes, one frame
Bytes presented to the decoder	144 bytes, nine formatter frames
Elements retained	35
Executed instructions reconstructed	114
Function blocks	6
Start-of-capture markers	2
Discontinuities within the traced execution	0
Executed instructions per byte produced by the device	0.89

The barrier is emitted by the extraction driver ahead of every payload, so it never occupied the sink and is counted apart from the trace the device produced. Both figures are given because the first states what the buffer held and the second states what the decoder consumed.

The last row answers what a buffer represents. At the density this capture exhibits, the two kilobytes of the sink hold on the order of eighteen hundred executed instructions. The figure is indicative, since the density depends on how often the program branches. It is also bounded by the format the sink stores, since a formatter frame of sixteen bytes carries at most fifteen bytes of trace.

The execution a full buffer holds is the interval immediately preceding a halt, a fault or a breakpoint, which is the interval a developer investigating a failure wants to see. A chosen function may be covered by placing a breakpoint at its end. Exposing the filtering capability of Section 5.5.6 would allow the buffer to cover a chosen region.

5.5.6 Limitations and Future Work

Limitations of the trace pipeline are stated in Table 5.6.

Further limitations concern the engine.

The two disassembly implementations have not been unified, so the view presented to the developer and the analysis used by the trace pipeline are produced by different code.

Several requests drive the debugger library outside the serialization of Section 4.3.5, which is the one remaining gap in the concurrency model.

Capabilities that depend on the direct path are unavailable when attaching to an existing session.

The frontend is not distributed, so installing the result requires building it.

Several directions extend the work, each removing one of the limitations stated above. Retaining the timing elements currently discarded would enable profiling from the same captures. Presenting exceptions explicitly would complete the reconstruction of execution across interrupt entry. Exposing the filtering capability of the trace unit would allow the buffer to cover a chosen region. Reading the component topology from the

Table 5.6: Limitations of the trace pipeline

Limitation	What it means
Component discovery	The trace source and the trace sink are located by name among the objects created by configuration, and their base addresses are stated there. Supporting a new device requires a configuration file to be written.
One protocol version	The drivers and the decoder handle the version implemented by the devices used. No device reachable by the on-chip route is lost, since every STM32 device carrying an on-chip trace sink implements that version, as Table 2.10 states.
One source and one sink	The first of each is selected, so a device offering several is not fully exploited. Tracing several cores is outside the scope of a proof of concept on a single-core target.
Timing discarded	Timestamps and cycle counts are decoded and dropped at the sink, so no timing analysis is possible without changing that component.
Exceptions not projected	The elements are retained and no consumer presents them, so an exception appears in the record as a change of address.
No trace filtering	The implementation traces without filtering, so the volume of trace is bounded by the size of the buffer alone. On these devices filtering would be driven from the processor comparators, which requires both those comparators and the trace unit to be modeled by the debug server.
Return classification	The analysis facility of the compiler library does not recognize the instruction forms this architecture emits for a return, so those forms are matched on the printed instruction.
Offline decoding not exposed	The stages require only a capture, a program image and the recorded register values. The standalone entry point that made this available during development was not carried forward.

ROM tables would remove the per-device configuration file.

5.6 Conclusion

This chapter has described the realization of the three deliverables. The application note assembles into one document the procedure for using instruction trace on these devices. The trace pipeline configures the components on the device, extracts a capture over the ordinary debug connection, decodes it against the program image, and reconstructs the executed instructions with their source locations. The debugger makes that pipeline usable during development and supplies the conventional capabilities alongside it.

Instruction trace on these devices requires no hardware beyond what an ordinary debug session already uses, provided the on-chip sink is driven correctly, which is the claim the application note exists to support. Reconstructing execution is not a decoding problem alone, since the trace states only which branches were taken and the program image supplies everything else.

Conclusion

The capability this project set out to reach was already present in the hardware. Instruction trace is implemented across a large part of the STM32 range, and on the families carrying a Trace Memory Controller a capture may be taken without leaving the chip. What was missing was a way to reach it with the tools and the probe an ordinary debug session already uses.

The work produced three results. An application note assembles into one document the procedure for using instruction trace on these devices, which the architecture specifications and the reference manuals state separately. A proof of concept demonstrates that a capture may be obtained from an on-chip sink, decoded against the program image, and reconstructed into the instructions that executed. A debugger carries that capability into a development tool, alongside breakpoints, variable inspection, memory and peripheral views, and disassembly.

Section 5.5 states what the result achieves. A capture of 128 bytes taken over the ordinary debug connection reconstructs 114 executed instructions, each attributed to a function, a source file and a line. No trace pins are used, and no probe beyond the one embedded on the development board.

Two conclusions carry beyond the project. The first is that the barrier to instruction trace on these devices was never the hardware. It was the distance between what the documentation states and what a developer must do, together with the absence of an open implementation to close that distance. The second is that reconstructing execution is not a decoding problem alone. The trace reports which branches were taken and nothing further, so the program image supplies everything else, and the result is only ever as good as the correspondence between the two.

What remains is stated where it belongs. The pipeline captures without filtering, discards the timing elements it decodes, and reads the trace topology from configuration rather than from the device. Each of these is a boundary of the implementation rather than of the approach, and Section 5.5.6 states what closing them would require.

Bibliography

- [1] STMicroelectronics, *ST company presentation*, [Accessed July 2025], 2024. [Online]. Available: https://www.st.com/resource/en/company_presentation/company_presentation.pdf.
- [2] M. Matraszek, M. Banaszek, W. Ciszewski, and K. Iwanicki, “FrankenTrace: Low-Cost, Cycle-Level, Widely Applicable Program Execution Tracing for ARM Cortex-M SoC,” in *Cyber-Physical Systems and Internet of Things Week 2023 (CPS-IoT Week Workshops '23)*, ACM, 2023. doi: 10.1145/3576914.3587521.
- [3] Z. Ning and F. Zhang, “Ninja: Towards Transparent Tracing and Debugging on ARM,” in *26th USENIX Security Symposium*, USENIX Association, 2017.
- [4] IAR Systems, *Requirements for Using ETM Trace, IAR Embedded Workbench for Arm*, [Accessed September 2026]. [Online]. Available: <https://docs.iar.com/ewarm/10.1x/en/c-spy-debugging/trace/introduction-to-using-trace/requirements-for-using-etm-trace.html>.
- [5] Microsoft, *Debug Adapter Protocol*, [Accessed August 2026]. [Online]. Available: <https://microsoft.github.io/debug-adapter-protocol/>.
- [6] Free Software Foundation, *Debugging with GDB: The GDB Remote Serial Protocol*, [Accessed August 2026]. [Online]. Available: <https://sourceware.org/gdb/current/onlinedocs/gdb.html/Remote-Protocol.html>.
- [7] The LLVM Project, *LLDB GDB Remote Protocol Extensions*, [Accessed August 2026]. [Online]. Available: <https://lldb.llvm.org/resources/lldb gdbremote.html>.
- [8] Arm Limited, *Arm Architecture Reference Manual, Armv7-M Architecture Profile*, Issue E.e, ARM DDI 0403E.e, Arm Limited, 2021.
- [9] Arm Limited, *Arm Architecture Reference Manual, Armv8-M Architecture Profile*, Version DDI0553B.z, Arm Limited, 2024.
- [10] C. Markusson, “Implementation of an application debugger for software in embedded systems,” Report no. LIU-IDA/LITH-EX-A--08/050--SE, M.S. thesis, Linköpings universitet, Department of Computer and Information Science, 2008.
- [11] DWARF Debugging Information Format Committee, *DWARF Debugging Information Format Version 5*, 2017. [Online]. Available: <https://dwarfstd.org/doc/DWARF5.pdf>.

- [12] Arm Limited, *Arm Embedded Trace Macrocell Architecture Specification, ETMv4.0 to ETMv4.6*, ARM IHI 0064H.b, Arm Limited, 2023.
- [13] Arm Limited, *ELF for the Arm Architecture*, AAELF32, ARM IHI 0044, Arm Limited, 2024. [Online]. Available: <https://github.com/ARM-software/abi-aa/blob/main/aaelf32/aaelf32.rst>.
- [14] Arm Limited, *CMSIS System View Description (CMSIS-SVD)*, [Accessed August 2026]. [Online]. Available: <https://open-cmsis-pack.github.io/svd-spec/main/index.html>.
- [15] Arm Limited, *Learn the Architecture: Introducing CoreSight Debug and Trace*, Issue 01, document 102520_0101_01_en, Arm Limited, 2025.
- [16] Arm Limited, *AMBA ATB Protocol Specification, ATBv1.0 and ATBv1.1*, ARM IHI 0032C, Arm Limited, 2012.
- [17] Arm Limited, *Arm Debug Interface Architecture Specification, ADIv5.0 to ADIv5.2*, ARM IHI 0031H, Arm Limited, 2022.
- [18] STMicroelectronics, *RM0477: STM32H7Rx/7Sx Arm-based 32-bit MCUs Reference Manual*, Rev 9, STMicroelectronics, 2025.
- [19] Arm Limited, *Understanding Trace: Trace Overview*, Version 2.0, document 102119_0200_00, Arm Limited, 2020.
- [20] Arm Limited, *Embedded Trace Macrocell Architecture Specification, ETMv1.0 to ETMv3.5*, ARM IHI 0014Q, Arm Limited, 2011.
- [21] Arm Limited, *Arm CoreSight Architecture Specification v3.0*, ARM IHI 0029G, Arm Limited, 2022.
- [22] Arm Limited, *Arm CoreSight SoC-400 Technical Reference Manual*, Issue 09, document 100536_0302_09_en, Arm Limited, 2023.
- [23] Arm Limited, *Arm CoreSight Trace Memory Controller Technical Reference Manual*, Revision r0p1, ARM DDI 0461B, Arm Limited, 2013.
- [24] STMicroelectronics, *AN4989: STM32 Microcontroller Debug Toolbox*, Rev 4, STMicroelectronics, 2026.
- [25] STMicroelectronics, *RM0433: STM32H742, STM32H743/753 and STM32H750 Advanced Arm-based 32-bit MCUs Reference Manual*, Rev 8, STMicroelectronics, 2023.
- [26] STMicroelectronics, *RM0090: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 Advanced Arm-based 32-bit MCUs Reference Manual*, Rev 19, STMicroelectronics, 2021.
- [27] Arm Limited, *Arm Cortex-M7 Processor Technical Reference Manual*, Revision r1p2, ARM DDI 0489D, Arm Limited, 2018.

- [28] STMicroelectronics, *STM32H7S3x8, STM32H7S7x8 Datasheet*, DS14359 Rev 6, STMicroelectronics, 2025.
- [29] STMicroelectronics, *UM3276: STM32H7Rx/7Sx Nucleo-144 Board (MB1737)*, Rev 2, STMicroelectronics, 2024.
- [30] Arm Limited, *Arm CoreSight ETM-M7 Technical Reference Manual*, Revision r0p1, ARM DDI 0494D, Arm Limited, 2016.
- [31] The OpenOCD Project, *Open On-Chip Debugger*, Version 0.12.0. [Accessed August 2026]. [Online]. Available: <https://openocd.org/>.
- [32] The LLVM Project, *The LLVM Target-Independent Code Generator: The MC Layer*, LLVM 22. [Accessed August 2026]. [Online]. Available: <https://llvm.org/docs/CodeGenerator.html>.
- [33] The LLVM Project, *The LLDB Debugger*, LLVM 22. [Accessed August 2026]. [Online]. Available: <https://lldb.llvm.org/>.
- [34] Linaro Limited, *OpenCSD: An Open Source CoreSight Trace Decode Library*, Version 1.8.3. [Accessed August 2026]. [Online]. Available: <https://github.com/Linaro/OpenCSD>.
- [35] D. Rath, “Open On-Chip Debugger: Design and Implementation of an On-Chip Debug Solution for Embedded Target Systems,” Diploma thesis, University of Applied Sciences Augsburg, 2005.

Abstract

This report describes a debugger that captures, decodes and presents the instruction trace of an STM32 microcontroller. Instruction trace records the path a program actually took through its code. The hardware producing it is present across much of the STM32 range, yet it is reachable only through proprietary tools and probes most developers do not have. Two CoreSight drivers were added to an open-source debug server, so that the trace unit and the on-chip trace buffer may be configured and drained over the ordinary debug connection. A pipeline decodes the captured bytes against the program image, reconstructs the executed instructions, attributes each to a function, a source file and a line, and presents the result inside a development tool during a live debug session. A capture of 128 bytes, drained from the on-chip buffer with no trace pins in use, reconstructs 114 executed instructions.

Keywords : Instruction trace, CoreSight, Embedded Trace Macrocell, Embedded debugging, STM32, Debug Adapter Protocol, OpenOCD.

Résumé

Ce rapport décrit un débogueur capable de capturer, de décoder et de présenter la trace d'instructions d'un microcontrôleur STM32. La trace d'instructions enregistre le chemin que le programme a réellement suivi dans son code. Le matériel qui la génère équipe une grande partie de la gamme STM32, mais il n'est exploitable qu'au moyen d'outils propriétaires et de sondes dont la plupart des développeurs ne disposent pas. Deux pilotes CoreSight ont été ajoutés à un serveur de débogage libre, afin que l'unité de trace et la mémoire tampon de trace intégrée au circuit puissent être configurées et vidées via la connexion de débogage habituelle. Une chaîne de traitement décode ensuite les octets capturés à l'aide de l'image du programme, reconstruit les instructions exécutées, rattache chacune à une fonction, à un fichier source et à une ligne, et présente le résultat au sein d'un environnement de développement, pendant une session de débogage. Une capture de 128 octets, extraite de la mémoire tampon intégrée au circuit et obtenue sans recourir aux broches de trace, permet de reconstruire 114 instructions exécutées.

Mots clés : Trace d'instructions, CoreSight, Embedded Trace Macrocell, Débogage embarqué, STM32, Debug Adapter Protocol, OpenOCD.